

SISTEMI DISTRIBUITI

2023-2024

Luca Giandomenico mat. 900162

Sistema distribuito:

Un sistema distribuito è un insieme di componenti hardware e software interconnessi da una rete che si coordinano attraverso la condivisione di memoria (message passing).

Ogni elemento è un nodo, ogni nodo è autonomo, ciò significa che opera alla velocità che vuole su task predefinite, non esiste un unico flusso di controllo.

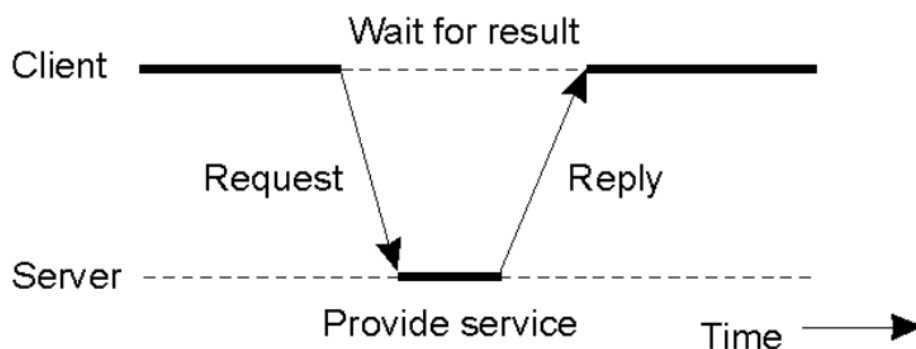
Il programma funziona indipendentemente da dove esso risiede o da dove risiedono i suoi dati. Un sistema distribuito ha la caratteristica di essere fault resistant, ovvero di poter continuare l'elaborazione anche se un nodo si dovesse bloccare.

Caratteristiche fondamentali:

1. Non esiste memoria condivisa, ma solo una comunicazione con scambio di messaggi, quindi non c'è stato globale.
2. Ogni componente è autonomo, ciò implica una esecuzione concorrente.
3. Non c'è un clock globale, né una possibilità di scheduling/controllo globale.
4. Ci sono fallimenti indipendenti dei singoli nodi.
5. L'esecuzione è concorrente.

Modello client server:

Il modello Client-Server prevede due componenti indipendenti:



L'architettura di base prevede che un client acceda ad un server con una richiesta e che il server risponda con un risultato.

Problemi fondamentali:

Generalmente un sistema distribuito deve far fronte a quattro problemi:

1. Identificare la controparte (naming): chi è?
2. Accedere alla controparte (access point): come accedo?
3. Come comunicare (communicating): come possono scambiarsi i messaggi i partecipanti? È necessario scegliere un protocollo.
4. Come capire il contenuto di un messaggio (communicating 2): è necessario scegliere una sintassi e una semantica.

Trasparenza:

Il concetto di trasparenza consiste nel nascondere i dettagli all'utente in modo tale che non possa influenzare il comportamento del sistema.

- **Naming:** si utilizzano dei nomi simbolici per accedere a delle risorse del sistema distribuito.
- **Access trasparenza:** metodo di accesso ad una risorsa in modo indipendente dalla sua posizione.
- **Location trasparenza:** nascondere la posizione di una risorsa nella rete.
- **Relocation trasparenza:** nascondere una risorsa che potrebbe essere spostata mentre è in uso.
- **Migration trasparenza:** nascondere il fatto che una risorsa possa spostarsi in un'altra posizione.
- **Replication trasparenza:** nascondere una risorsa replicata.
- **Concurrency trasparenza:** nascondere il fatto che una risorsa possa essere condivisa da più utenti.
- **Failure trasparenza:** nascondere il fallimento e il ripristino di una risorsa.
- **Persistence trasparenza:** nascondere il fatto che una risorsa sia volatile oppure salvata permanentemente.

Architetture software:

Un'architettura software definisce la struttura del sistema, le interfacce tra i componenti e i pattern di interazione. I sistemi distribuiti possono essere organizzati secondo diversi stili architetturali:

1. Architetture a strati (layered): per i sistemi operativi o il middleware.
2. Architetture a livelli (tier): per le applicazioni client-server.
3. Architetture basate sugli oggetti.
4. Architetture centrate sui dati: il web come file system condiviso.

Architetture stratificate:

Una architettura a strati è un tipo di architettura software che organizza il software in diversi strati. Ogni strato è costruito sopra un altro più generale. **Gli strati superiori sono più specifici per le applicazioni mentre quelli inferiori sono più generali.**

Sistemi operativi distribuiti:

Sono i sistemi operativi che governano i sistemi distribuiti. Esistono tre modelli principali, con diversi obiettivi:

- **DOS: distributed operating system**, è un software che controlla diverse macchine e coordina le loro risorse per fornire agli utenti un singolo ambiente computazionale.
- **NOS: network operating system**, è un software che connette diversi dispositivi e computer alla rete e permette loro di condividere risorse sulla rete.
- **Middleware:** si tratta di uno strato aggiuntivo, posto in cima al NOS che implementa dei servizi generali.

Servizi del middleware:

Il middleware fornisce diversi servizi che possono affrontare sia problemi generali, ma anche problemi specifici. I Servizi di base sono:

- **Naming:** Utilizza nomi simbolici per identificare le entità distribuite. I registri come DNS o registri RMI mappano nomi simbolici a indirizzi reali.
- **Accesso Trasparente:** Fornisce un modello di comunicazione che nasconde le complessità del passaggio dei messaggi.
- **Persistenza:** Offre servizi di archiviazione automatica dei dati su sistemi di file o database.

- **Transazioni Distribuite:** Garantisce la coerenza nelle operazioni di lettura/scrittura tra database.
- **Sicurezza:** Definisce modelli per proteggere l'accesso ai dati e ai servizi con diversi livelli di permesso.

Politiche e meccanismi:

Nel contesto dei sistemi distribuiti, è essenziale che i componenti operino in modo indipendente, fornendo servizi autonomamente mentre collaborano per svolgere compiti più complessi. Questa collaborazione richiede una chiara distinzione tra politiche e meccanismi:

- I meccanismi rappresentano le capacità dei componenti.
- Le politiche definiscono come le capacità vengono utilizzate per modellare il comportamento.

Maggiore è il grado di separazione tra politica e meccanismo, più è necessario fornire meccanismi complessi portando ad una gestione più difficile del sistema.

Protocolli:

I protocolli definiscono il formato, l'ordine di invio e di ricezione dei messaggi tra i dispositivi, il tipo dei dati e le azioni da eseguire quando si riceve un messaggio.

Servizi di trasporto internet:

Servizio TCP:

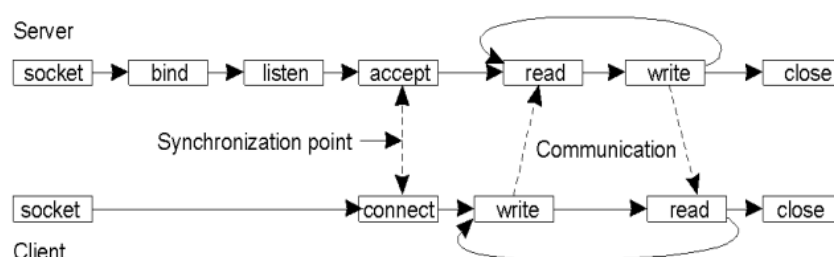
- Orientato alla connessione.
- Trasporto affidabile.
- Controllo di flusso e congestione
- Non offre garanzia di banda e ritardi minimi.

Nel servizio TCP, ogni messaggio viene scomposto in segmenti e numerato per garantire il riordinamento dei dati e controllare duplicazioni e perdite. È il protocollo prediletto per i sistemi distribuiti.

Servizio UDP:

UDP presenta un trasporto non affidabile tra processi mittente e ricevente, non offre alcuna funzionalità aggiuntiva; tuttavia, può essere utile per le applicazioni che tollerano una perdita parziale dei dati a vantaggio delle prestazioni, ad esempio audio e video.

In entrambi i servizi, ci sono problemi fondamentali che vanno affrontati per definire una corretta comunicazione. È necessario gestire il ciclo di vita delle parti comunicanti, ovvero determinare come vengono attivati e terminati il client e il server. Ci deve essere una ripartizione dei compiti tra client e server sebbene abbiano ruoli pressoché ben separati, a volte è necessario, a seconda dell'applicazione e dell'influenza sulle prestazioni, definire cosa è necessario che faccia il client e cosa il server.



API socket system calls:

Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket, set the queue length
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Write	Send some data over the connection
Read	Receive some data over the connection
Close	Release the connection

Socket:

Ogni processo comunica attraverso canali, che gestiscono flussi di dati in ingresso e in uscita. Le socket sono particolari canali per la comunicazione tra processi che non condividono memoria, per esempio perchè risiedono su macchine diverse.

La comunicazione via socket avviene attraverso flussi di byte, dopo una connessione esplicita, tramite normali system call read e write. Le chiamate read/write sono chiamate sospensive che bloccano il processo finché il sistema operativo non ha effettuato la lettura o la scrittura.

Processi e socket:

Il server crea una socket collegata alla well-known port, che identifica il servizio fornito, dedicata a ricevere richieste di connessione. Con la primitiva accept(), il server crea una nuova socket, cioè un nuovo canale, dedicato alla comunicazione con il client.

Read e write:

Le socket trasportano stream, ovvero flussi di bytes, quindi non esiste il concetto di messaggio, il flusso è continuo e senza fine. La lettura e la scrittura avvengono per un numero arbitrario di byte, si devono quindi prevedere cicli di lettura che termineranno in base alla dimensione dei messaggi come stabilito dal formato del protocollo applicativo in uso.

Le socket in Java:

Java definisce alcune classi che costituiscono un'interfaccia ad oggetti alle system call precedentemente illustrate. Le principali sono **java.net.Socket** e **java.net.ServerSocket**.

Costruttori:

Crea una socket non connessa.

```
• public Socket()
```

Crea una socket e la connette ad una specifica porta sull'host specificato. se l'host specificato è null, il socket viene connesso all'indirizzo di loopback. Viene lanciata un'eccezione se non si riesce a risalire all'indirizzo ip dell'host.

```
• public Socket(String host, int port) throws UnknownHostException, IOException
```

Crea una socket di tipo stream che viene connessa alla porta specificata e all'indirizzo ip specificato.

```
• public Socket(InetAddress, address, int port) throws IOException
```

Crea un serverSocket senza associarlo a un indirizzo e a una porta specifici.

```
• public ServerSocket() throws IOException
```

Crea un serverSocket associato ad una specifica porta.

```
• public ServerSocket(int port) throws IOException
```

Crea un serverSocket e lo collega ad un numero specifico di porta, con uno specifico (backlog) – ovvero la lunghezza massima della coda delle connessioni in attesa di essere accettate dal serverSocket.

```
• public ServerSocket(int port, int backlog) throws IOException
```

Gestione delle connessioni:

Collega una socket ad un indirizzo locale, se l'indirizzo specificato è null, il sistema sceglierà automaticamente una porta effimera e un indirizzo locale valido per associare la socket.

```
• public void bind(SocketAddress bindpoint) throws IOException
```

Connette la socket al server

```
• public void connect(SocketAddress endpoint) throws IOException
```

Connette la socket al server con un valore di time-out specifico espresso in millisecondi.

```
• public void connect(SocketAddress endpoint, int timeout) throws IOException
```

Chiude la socket:

```
• public void close().
```

Rimane in ascolto e accetta una connessione, restituisce la nuova socket.

```
• public Socket accept()throws IOException.
```

Gestione dei flussi:

Restituisce l'input stream legato alla socket.

```
• public InputStream getInputStream()throws IOExceptionpublic InputStream getInputStream()throws IOException.
```

Restituisce l'output stream legato alla socket per scrivere bytes.

```
• public OutputStream getOutputStream()throws IOException.
```

Metodi di utilità:

Ritorna l'indirizzo locale del serverSocket oppure null se il serverSocket non è connesso.

```
• public InetAddress getInetAddress()
```

Ritorna la porta dalla quale la socket sta ascoltando, altrimenti -1 se la socket non è ancora collegata.

```
• public int getLocalPort()
```

Ritorna l'indirizzo dell'endpoint al quale la socket è collegata, oppure null se questa non è collegata.

```
• public SocketAddress getLocalSocketAddress()
```

Progettazione di un'applicazione:

Per quanto riguarda il lato client, l'architettura è più semplice, è spesso un'applicazione convenzionale che usa una socket anziché un altro canale di I/O. Non ci sono particolari problemi di sicurezza, ha effetti solo sul lato client.

Invece, il lato server, prevede un'architettura generale in cui viene creata una socket con una porta nota per accettare le richieste di connessione. Inoltre, si prevede un ciclo infinito in cui alternare.

1. Attesa/accettazione di una richiesta di connessione da un client.
2. Ciclo lettura-esecuzione, invio risposta fino al termine della conversazione (stabilito spesso dal client).
3. Chiusura connessione.

Problematiche connesse:

L'affidabilità del server è strettamente dipendente dall'affidabilità della comunicazione tra lui e i suoi client.

Architetture dei server:

Introducendo le architetture dei server, è doveroso distinguerli in quattro tipologie:

1. **Iterativi:** soddisfano una richiesta alla volta.
2. **Concorrenti processo singolo:** simulano la presenza di un server dedicato.
3. **Concorrenti multi-processo:** creano server dedicati.
4. **Concorrenti multi-thread:** creano thread dedicati.

I/O non bloccante:

Le operazioni di lettura e scrittura comportano l'uso di system call bloccanti, le system call bloccanti comportano l'attesa della conclusione dell'operazione richiesta prima della restituzione del controllo al chiamante.

L'I/O non bloccante è una tecnica di gestione delle operazioni di input/output che consente a un programma di eseguire altre operazioni mentre aspetta che un'operazione di I/O sia completata, senza bloccare il thread principale. Per leggere in modo non bloccante è necessario verificare se il canale sia pronto prima di effettuare un'operazione di lettura o scrittura. È possibile fare ciò tramite la system call `select()` che permette di gestire in modo non bloccante i diversi canali I/O, in particolare, sospende il processo finché non è possibile fare un'operazione di I/O. In Java, i canali (channels) nell'I/O non bloccante consentono di gestire più connessioni simultanee con un singolo thread, i canali possono essere impostati in modalità non bloccante sotto la supervisione di un selector. Un Selector è in grado di riconoscere quando uno o più canali diventano disponibili per il trasferimento di dati.

Java Selector:

Un Selector permette di gestire dei `SelectableChannel` con una `select()` in Java. Un selettore può essere creato invocando il metodo statico `open()` della classe `Selector`.

```
Selector selector = Selector.open();
```

I canali da monitorare con la select devono essere

1. messi in modalità non bloccante, e poi
2. registrati con il metodo `register()`

```
channel.configureBlocking(false);
```

```
SelectionKey key = channel.register(selector, SelectionKey.OP_READ);
```

Ogni canale può essere registrato per monitorare quattro tipo di eventi (modi di utilizzo) definiti da costanti nella classe `SelectionKey`

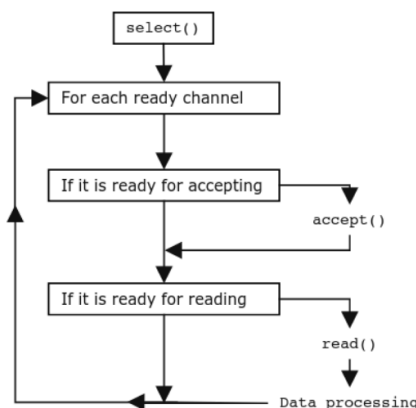
1. **Connect:** when a client attempts to connect to the server `SelectionKey.OP_CONNECT`
2. **Accept:** when the server accepts a connection from a client `SelectionKey.OP_ACCEPT`
3. **Read:** when the server is ready to read from the channel `SelectionKey.OP_READ`
4. **Write:** when the server is ready to write to the channel `SelectionKey.OP_WRITE`

Server iterativo:

Al momento di una richiesta di connessione, il server crea una socket temporanea per stabilire una connessione diretta con il client. Le eventuali ulteriori richieste di connessione per il server verranno accodate alla porta nota per essere successivamente soddisfatte. Il vantaggio del server iterativo è il fatto di essere semplice da progettare, d'altra parte risulta inefficiente in quanto viene servito un cliente alla volta.

Server concorrente:

Un server concorrente può gestire più connessioni client, attraverso la possibilità di simulare più socket utilizzando un solo processo, oppure creando processi slave.



Server multiprocesso:

Un server multiprocesso non è altro che un server concorrente che crea nuovi processi slave attraverso la primitiva `fork()`. La primitiva `fork` crea un processo clone del padre che eredita i canali di comunicazione ed esegue lo stesso codice. Il codice deve prevedere che il padre chiuda la socket per la conversazione con il client e che il figlio chiuda a sua volta la socket per l'accettazione di nuove connessioni. La struttura del server è la stessa di quella iterativa in quanto ogni server gestisce un solo client.

Confronto tra modelli:

Per confrontare i modelli, ovvero chiarire quando è necessario un server monoprocesso e quando multiprocesso, è più efficace confrontare le loro caratteristiche principali.

Monoprocesso: i server monoprocesso sono iterativi e concorrenti quindi:

- Gli utenti condividono lo stesso spazio di lavoro.
- Un server monoprocesso è capace di gestire richieste di letture e scritture su dati condivisi da parte di applicazioni in cui più parti devono collaborare per modificare quei dati.

Multiprocesso: nel caso dei server multiprocesso:

- Ogni utente ha uno spazio di lavoro autonomo.
- Un server multiprocesso è in grado di gestire richieste da parte di più utenti o processi, ma è principalmente configurato per permettere solo operazioni di lettura sui dati condivisi.
- Il server multiprocesso è flessibile e può supportare sia applicazioni cooperative che lavorano su dati condivisi sia applicazioni autonome che gestiscono i propri dati.

Concorrenza e programmazione multithreading:

La concorrenza è la contemporaneità di esecuzione di parti diverse di uno stesso programma, ovvero la capacità di far progredire più di una attività simultaneamente. La concorrenza è diversa dal parallelismo, che è invece la capacità di eseguire più di una attività simultaneamente.

- Single core + multiprogrammazione = concorrenza senza parallelismo.
- Multicore + multiprogrammazione = concorrenza con il parallelismo.

Parallelismo delle attività e dei dati:

- **Parallelismo dei dati:** diversi core eseguono la stessa operazione, ovvero eseguono in parallelo attività identiche ma distinte.
- **Parallelismo delle attività:** diversi core eseguono attività diverse in parallelo.

Legge di Amdhal:

Fornisce il guadagno in termini di performance derivante dall'aggiunta di core ad un'applicazione che ha componenti sia sequenziali che parallele.

Programmazione concorrente:

La programmazione concorrente è un paradigma di programmazione che permette l'esecuzione simultanea di più processi o sottoprocessi (thread) all'interno di un'applicazione. Questi processi possono lavorare insieme per eseguire un'attività specifica e possono interagire tra loro scambiandosi messaggi o accedendo a strutture dati condivise. La programmazione concorrente mira a sfruttare al massimo le capacità di elaborazione dei moderni sistemi multi-core e multi-processore e viene impiegata per evitare di bloccare l'intera esecuzione di un'applicazione a causa dell'attesa del completamento di un'azione di I/O.

Processi:

Un sistema operativo esegue un certo numero di programmi sullo stesso sistema di elaborazione, il numero di programmi da eseguire può essere arbitrariamente elevato, di solito molto maggiore del numero di CPU del sistema. A tale scopo il sistema operativo realizza e mette a disposizione un'astrazione detta processo. Un processo è un'entità attiva astratta definita dal sistema operativo allo scopo di eseguire un programma.

Differenza tra programma e processo:

- **Programma:** il programma è un'entità passiva, un insieme di istruzioni contenute in un file sorgente o in un eseguibile.
- **Processo:** un processo è un'entità attiva, un esecutore di programma o un programma in esecuzione. Un processo viene caricato in memoria e attivato e gli vengono associate strutture ed informazioni necessarie al funzionamento. Un processo contiene almeno un flusso di controllo in esecuzione.

Uno stesso programma può dare origine a diversi processi, diversi utenti possono eseguire lo stesso programma e uno stesso programma viene eseguito più volte, anche contemporaneamente, dallo stesso utente.

Multiprogrammazione e multitasking:

Gli obiettivi principali del sistema operativo sono:

- Massimizzare l'utilizzo della CPU, ovvero mantenere la CPU impegnata nell'esecuzione dei programmi il maggior tempo possibile.
- Dare l'illusione che ogni processo abbia una CPU dedicata.

Due tecniche adottate nei sistemi operativi sono la multiprogrammazione e il multitasking.

- **Multiprogrammazione:** la multiprogrammazione consiste nel mantenere più programmi in memoria e impedire ad un programma che non è in grado di proseguire l'esecuzione di mantenere la CPU.
- **Multitasking:** il multitasking consiste nel far sì che un programma interattivo reagisca agli input utente in un tempo accettabile, si tratta quindi di un'estensione della multiprogrammazione in cui la CPU viene sottratta periodicamente al programma in esecuzione ed assegnata ad un altro programma, in questo modo tutti i programmi progrediscono in maniera continuativa nella propria esecuzione.

Operazioni sui processi:

I sistemi operativi forniscono delle chiamate di sistema con le quali un processo può creare, terminare, manipolare altri processi. Dal momento che solo un processo può creare un altro processo, all'avvio il sistema operativo crea dei processi primordiali dai quali tutti i processi utente e di sistema vengono progressivamente creati.

Struttura di un processo:

Un processo è composto da diverse parti:

- Lo stato dei registri del processore; che esegue il programma, incluso il program counter.
- Lo stato della regione di memoria centrale usata dal programma, o immagine del processo.
- Lo stato del processo stesso (new, running, waiting, ready, terminated).
- Le risorse del sistema operativo in uso al programma.

Immagine di un processo:

L'immagine di un processo è formata da:

- Il codice del programma (code section).
- La data section, contenente le variabili globali.

- Lo stack delle chiamate, contenente parametri, variabili locali e indirizzo di ritorno.
- Lo heap, contenente la memoria allocata dinamicamente durante l'esecuzione.

Code e data section hanno dimensioni costanti per tutta la vita del processo mentre Stack e heap invece crescono / decrescono durante la vita del processo.

Commutazione di contesto:

Quando la CPU deve passare dall'esecuzione di un processo a quella di un altro processo avviene una commutazione di contesto (context switch). La commutazione di contesto viene effettuata dal dispatcher, che salva il contesto (stato, registri...) del processo da interrompere nel suo PCB, successivamente carica il contesto del processo da eseguire dal suo PCB. Il tempo necessario per il context switch è overhead, tempo perso, non viene eseguito alcun lavoro utile.

Multithreading:

Se supponiamo che un processo possa avvalersi di molti processori virtuali, più istruzioni possono essere eseguite in maniera concorrente, e quindi il processo può avere più flussi, thread, di esecuzione concorrenti.

Per poter consentire l'esecuzione di diversi flussi di controllo in parallelo sono stati introdotti i thread. I thread di uno stesso processo condividono la memoria globale (data and heap), la memoria contenente il codice, code section e le risorse ottenute dal sistema operativo (ad esempio i file aperti). Ogni thread di uno stesso processo però deve avere un proprio stack, altrimenti le chiamate a subroutine di un thread interferirebbero con quelle di un altro thread concorrente.

Vantaggi del multithreading:

- **Condivisione di codice e dati:** i thread di un processo condividono lo spazio di memoria del processo.
- **Economia:** creare un thread richiede meno risorse che creare un processo ed il cambio di contesto è più rapido.
- **Migliori tempi di risposta:** l'applicazione può eseguire un'operazione lunga in un thread a parte e continuare a rispondere agli input utente.
- **Scalabilità:** il multithreading in una macchina con più processori aumenta il grado di parallelismo di un'applicazione.
- **Ottimizzazione delle risorse:** se un thread è in attesa di una operazione di I/O, un altro thread del medesimo programma può accedere alla CPU evitando il context switch dell'intero processo.

Thread a livello utente:

I thread a livello utente sono quelli disponibili nello spazio utente dei processi e vengono offerti dalle librerie thread ai processi.

Thread a livello del kernel:

I thread a livello kernel sono quelli implementati nativamente dal kernel e vengono utilizzati per strutturare il kernel stesso in maniera concorrente.

Modelli di supporto multithreading:

- **Molti a uno:** i thread a livello utente di un certo processo sono implementati su un solo thread a livello del kernel. Il vantaggio del modello uno a uno consiste nel fatto che risulta utilizzabile

su ogni sistema operativo, d'altra parte se un thread a livello utente esegue una chiamata bloccante, questa comporterà il blocco di tutti i thread utente dello stesso processo.

- **Uno a uno:** ogni thread a livello utente è implementato su un singolo, distinto thread a livello del kernel. Il modello uno a uno permette un maggior grado di concorrenza, ma anche di sfruttare il parallelismo nei sistemi multicore.
- **Molti a molti:** i thread a livello utente di un certo processo sono implementati su un insieme di thread a livello del kernel in numero inferiore. L'associazione thread utente/thread kernel è dinamica e viene stabilita da uno scheduler interno alla libreria di thread. Il modello molti a molti cerca di combinare i vantaggi dei modelli molti a uno e uno a uno, risulta complesso nell'implementazione in quanto la libreria di thread deve dinamicamente alternare l'esecuzione dei thread a livello utente sui thread a livello del kernel disponibili)

Lightweight process:

Un lightweight process è l'interfaccia offerta dal kernel alle librerie dei thread per usare i thread del kernel. Ogni lightweight process è un oggetto astratto associato staticamente dal kernel ad esattamente un thread del kernel.

Multithreading in Java:

I threads in java sono astrazioni fornite e gestite dalla Java virtual machine, sono implementati sfruttando il modello di threading offerto dal sistema operativo sottostante. La JVM offre una libreria per la definizione dei thread e per l'interazione del sistema operativo per la loro esecuzione.

I thread in Java possono essere creati principalmente in due modi:

1. Estensione della classe standard `java.lang.Thread`.
2. Tramite l'implementazione del metodo `run()` con l'interfaccia `java.lang.Runnable`.

Il thread Main:

In Java ogni programma in esecuzione è un thread, il metodo `main()` è associato al thread `main`. Per poter accedere alle proprietà del thread in esecuzione è necessario ottenerne un riferimento tramite il metodo `currentThread()`.

```
public class ThreadMain {
    public static void main(String args[]) {
        Thread t = Thread.currentThread();
        System.out.println("Thread corrente: " + t );

        t.setName("Mio Thread");
        System.out.println("Dopo cambio nome: " + t );
    }
}
```

Classe principale per I thread in Java:

Il metodo più semplice per creare ed eseguire un thread in java è il seguente:

1. Estendere la classe `java.lang.Thread` che contiene un metodo `run()` vuoto.
2. Ridefinire il metodo `run()` nella sottoclasse (override):
 - a. Il codice eseguito dal thread è incluso nel metodo `run()` e nei metodi invocati direttamente o indirettamente da `run()`.
 - b. Questo è il codice che verrà eseguito.
3. Creare un'istanza della sottoclasse.
4. Richiamare il metodo `start()` sulla sottoclasse.

Fare partire i thread con il metodo start():

Tramite una chiamata al metodo `start()` il thread è reso pronto all'esecuzione, il thread verrà mandato in esecuzione attraverso il metodo `run()` dallo scheduler quando questo lo riterrà opportuno. Il metodo `run()` non deve essere chiamato esplicitamente, la sua invocazione avviene in maniera automatica nella classe `Thread`. I due thread saranno eseguiti in modo concorrente ed indipendente. L'ordine con cui ogni thread eseguirà le proprie istruzioni è noto, ma l'ordine globale in cui le istruzioni dei vari thread saranno eseguite effettivamente è indeterminato.

Interfaccia runnable:

In Java, poiché non è possibile utilizzare l'ereditarietà multipla, è comune utilizzare l'interfaccia `Runnable` per implementare thread. Questo approccio offre maggiore flessibilità poiché consente di estendere altre classi di base.

Utilizzo dell'Interfaccia `Runnable` per creare thread:

- Definire una classe che implementa `Runnable`, fornendo una significativa implementazione del metodo `run()`.
- Creare un'istanza di questa classe.
- Creare un nuovo oggetto `Thread`, passando l'istanza della classe implementante `Runnable` al costruttore.
- Avviare il thread chiamando il metodo `start()` sull'istanza di `Thread`.

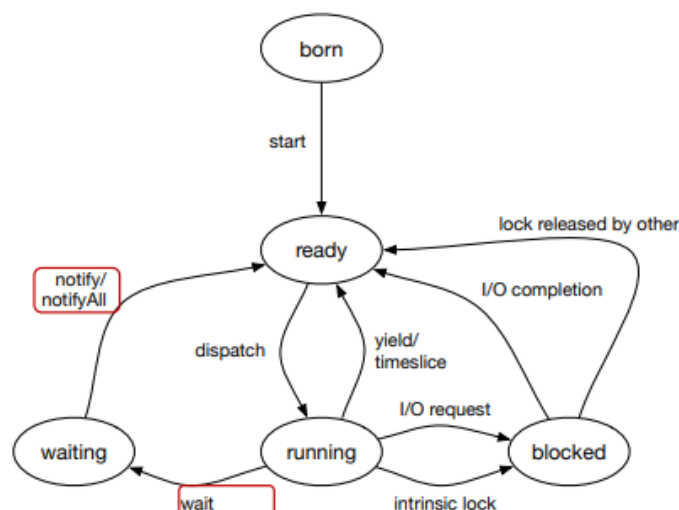
Thread alive o terminated?

L'invocazione del metodo `start()` porta il Thread ad eseguire il metodo `run()`, un thread è considerato alive finché il metodo `run()` non termina, quando `run()` ritorna, il thread è considerato terminated. Una volta che un thread è terminato, questo non può essere rieseguito. In java esiste il metodo `isAlive()` che può essere usato per valutare se il thread sia stato fatto partire e al contempo non sia stato terminato, ovvero se il metodo `run()` non sia terminato.

Diagramma degli stati di un thread:

Quando viene invocato il metodo `start()` su un thread, il thread non viene eseguito immediatamente, ma passa nello stato di ready, quando lo scheduler seleziona il thread dalla coda, questo passa nello stato running, ed esegue il metodo `run()`.

`yield()` fornisce un meccanismo per informare lo scheduler che il thread corrente è disposto a rinunciare al suo attuale uso del processore ma vorrebbe essere schedulato di nuovo il prima possibile.



Cancellazione dei thread:

La cancellazione di un thread corrisponde alla sua prematura eliminazione e può avvenire in due modi diversi:

1. **Cancellazione immediata:** il thread è terminato immediatamente.
2. **Cancellazione differita:** il thread controlla periodicamente se deve terminare, in modo da effettuare una terminazione ordinata, in Java funziona così.

Come fermare un thread in Java?

Per interrompere un thread in Java, è possibile chiamare il metodo `interrupt()` sul thread che si desidera fermare. Ciò imposta il flag di interruzione del thread. Tuttavia, è importante notare che ciò non interrompe immediatamente l'esecuzione del thread. Invece, è responsabilità del codice all'interno del thread controllare periodicamente lo stato di interruzione e terminare l'esecuzione in modo sicuro quando viene rilevata un'interruzione.

Programmi concorrenti e sequenziali:

I programmi concorrenti si distinguono notevolmente dai più comuni programmi sequenziali. In un programma sequenziale (deterministico), l'esecuzione ripetuta con lo stesso input produce sempre lo stesso risultato, rendendo eventuali bug riproducibili. Questo non vale per i programmi concorrenti, dove il comportamento di un thread può dipendere in modo non deterministico dagli altri thread.

Le interazioni tra gli agenti concorrenti possono avvenire in diversi modi:

- La **cooperazione**, con interazioni prevedibili e desiderate che sono essenziali per la logica del programma, ciò avviene tramite lo scambio di semplici informazioni come segnali, attraverso sincronizzazione diretta o esplicita.
- La **competizione** si verifica quando gli agenti competono per accedere a una risorsa condivisa, richiedendo politiche di accesso alla risorsa e sincronizzazione indiretta o implicita.
- Le **interferenze** si riferiscono a interazioni "non prevedibili e non desiderate", spesso causate da errori di programmazione dipendenti dalle tempistiche e difficili da riprodurre.

Meccanismi di sincronizzazione:

I meccanismi di controllo dell'ordine relativo delle attività dei processi/thread sono fondamentali per garantire un corretto funzionamento dei programmi concorrenti. Se si adotta un modello di memoria condivisa, ci sono due principali strategie:

1. La **mutua esclusione**, che impedisce l'accesso simultaneo dei dati da parte di più thread, mediante la creazione di regioni critiche del codice.
2. La **sincronizzazione su condizione**: prevede di sospendere l'esecuzione di un thread fino al verificarsi di una condizione opportuna sulle risorse condivise.

Nel caso di un modello a scambio di messaggi, ci sono altri due approcci:

1. La **sincronizzazione su eventi**: prevede di sospendere l'esecuzione di un thread fino alla verifica di un evento specifico.
2. Le primitive di invio (`send`) e ricezione (`receive`) di solito incorporano implicitamente un meccanismo di sincronizzazione. In particolare, quando un thread invia un messaggio, è implicitamente garantito che il thread ricevente non possa ricevere quel messaggio finché non sia stato effettivamente inviato.

Questi meccanismi sono offerti dal sistema operativo e, in Java, sono implementati a livello di JVM per i thread.

Sincronizzazione su eventi:

Il metodo `join()` della classe `Thread` è un meccanismo per sincronizzare temporalmente due thread. Quando viene invocato su un thread, il thread chiamante entra in uno stato di attesa fino a quando il thread chiamato non termina la sua esecuzione.

Sincronizzazione su condizione: barriere

Il meccanismo di sincronizzazione prevede che i thread siano messi in attesa progressivamente fino a quando una condizione globale non viene soddisfatta. Ad esempio, in un compito comune suddiviso in più fasi, ogni fase viene assegnata a più thread. Ogni thread esegue il suo compito e attende che tutti gli altri thread abbiano completato prima di procedere alla fase successiva.

Una possibile implementazione di questo approccio coinvolge l'uso di un contatore condiviso che tiene traccia del numero di processi che devono terminare. Questo contatore viene decrementato ogni volta che un thread completa il suo compito.

Race conditions:

Nei programmi concorrenti molti problemi sono causati dalle race conditions, ovvero quelle situazioni in cui thread diversi operano su una risorsa comune e il risultato dipende dall'ordine in cui essi effettuano le loro operazioni. Questo fenomeno si manifesta quando i thread competono per l'accesso a una risorsa condivisa, come una variabile in memoria. In Java, le race condition possono manifestarsi quando più thread accedono e modificano variabili condivise in modo asincrono e non coordinato.

Implementare la sincronizzazione:

La sincronizzazione implica che almeno un thread sia in attesa, che sia per una risorsa o per l'esecuzione di un altro thread. Ci sono due approcci comuni per gestire questa attesa:

1. **Attesa attiva (busy waiting o spinning):** questo approccio ha senso principalmente nei sistemi multiprocessore, non richiede un cambio di contesto e consiste nel verificare periodicamente se una certa condizione è valida. Uno svantaggio dell'attesa attiva è il tempo che viene sprecato dalla CPU.
2. **Sincronizzazione basata sullo scheduler:** In questo caso, il thread si sospende in modo attivo o automatico fino a quando non si verifica un evento specifico. Quando l'evento atteso si verifica, il thread viene risvegliato e può riprendere la sua esecuzione.

Sincronizzazione basata su sezione critica:

La sezione critica è un blocco di codice che può essere eseguito da un solo thread alla volta, mirando a garantire la mutua esclusione. L'obiettivo è stabilire un protocollo di cooperazione tra i processi tale che, quando un thread è all'interno della sua sezione critica, gli altri non possano accedervi contemporaneamente. Questo protocollo prevede che ogni thread acceda alla risorsa condivisa tramite tre operazioni principali:

1. Una sezione di ingresso (per richiedere l'accesso alla sezione critica).
2. La sezione critica stessa.
3. Una sezione di uscita (per eseguire azioni come la notifica degli altri processi).

Lock:

Il Lock è una variabile logica che può essere manipolata atomicamente. Si tratta di una variabile binaria che offre due metodi principali: `lock()` e `unlock()`, che consentono di acquisire e rilasciare il lock in modo atomico. Una volta che un thread ha acquisito un lock, gli altri thread che richiedono il lock si bloccano fino a quando il thread che lo detiene non lo rilascia. Per ogni lock è necessario gestire una coda di thread in attesa.

```
import java.util.concurrent.locks.*;

public class SharedRes {
    private Lock l = new ReentrantLock();
    private int counter = 0;

    public void increment() {
        l.lock(); // B
        try {
            // doing some action
            try {
                counter++;
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
        } finally {
            // Release the permit.
            l.unlock(); // B
        }
    }
}
```

Semaforo:

Un semaforo è una variabile intera che può essere manipolata solo tramite due operazioni atomiche: `acquire()` e `release()`. La prima operazione attende che la variabile sia maggiore di zero e quindi la decrementa, mentre la seconda incrementa incondizionatamente la variabile. È necessario gestire una coda di thread in attesa per ogni semaforo.

```
import java.util.concurrent.*;

public class MaxAccessRes {
    private Semaphore sem;
    public MaxAccessRes(int size) {
        sem = new Semaphore(size);
    }
    public void access() {
        try {
            sem.acquire(); // B
            // doing some action
            Thread.sleep(1000);
        } catch (InterruptedException exc) {}
        // Release the permit.
        sem.release(); // B
    }
}
```

Mutua esclusione implicita in oggetti java: metodi synchronized

L'attivazione di metodi Java può essere resa mutuamente esclusiva tramite la specifica della parola chiave `synchronized`. Java associa un lock intrinseco/implicito ad ogni oggetto che abbia almeno un metodo `synchronized`. Poiché ogni oggetto in java estende la classe `Object`, ogni oggetto ha un suo lock.

Quando un thread, come T1, esegue un metodo o blocco `synchronized`, gli altri thread che richiedono l'esecuzione dello stesso o di un altro metodo `synchronized` vengono messi in attesa fino a quando T1 completa l'esecuzione del metodo.

Attivazione dei metodi **synchronized**:

L'attivazione di metodi Java può essere resa mutuamente esclusiva tramite l'utilizzo della parola chiave **synchronized**. Java associa un lock intrinseco o implicito a ogni oggetto che abbia almeno un metodo sincronizzato. Poiché ogni oggetto in Java estende la classe **Object**, ogni oggetto ha il suo lock. Un oggetto con un metodo **synchronized** è considerato la risorsa condivisa tra i thread. Tuttavia, non è possibile utilizzare **synchronized** su un costruttore o su singoli campi. Quando un thread esegue un metodo o un blocco **synchronized**, gli altri thread che richiedono l'esecuzione dello stesso o di un altro metodo o blocco **synchronized** vengono messi in attesa finché il thread non completa la sua esecuzione.

```
public class SynchronizedCounter {
    private int c = 0;

    public synchronized void increment() { //acquisisce il lock
        c++; // rilascia il lock
    }

    public synchronized void decrement() { //acquisisce il lock
        c--; // rilascia il lock
    }

    public synchronized int value() { //acquisisce il lock
        return c; // rilascia il lock
    }
}
```

Ereditarietà **synchronized**:

La parola chiave **synchronized** in Java non fa parte propriamente della segnatura di un metodo, il che significa che una classe derivata può decidere di ridefinire un metodo come **synchronized** o rimuovere l'attributo **synchronized** da un metodo ereditato. Questa caratteristica offre una notevole flessibilità, poiché consente di definire classi per un utilizzo sequenziale senza la necessità di considerare i problemi di concorrenza. In seguito, queste classi possono essere adattate per un utilizzo concorrente derivando sottoclassi e aggiungendo la sincronizzazione necessaria tramite **synchronized**.

Monitor:

Un monitor è una primitiva a un livello superiore rispetto ai lock e ai semafori, si tratta di un tipo di dati astratto le cui variabili interne sono accessibili solo da un insieme di procedure esposte dal monitor. Solo un processo o thread alla volta può essere attivo nel monitor, garantendo la mutua esclusione.

```
monitor MonitorName {
    // Dichiarazioni variabili condivise
    int sharedVariable1;
    int sharedVariable2;

    // Definizione delle funzioni del monitor
    function void P1(...) {
        // Codice della funzione P1
    }
    function void P2(...) {
        // Codice della funzione P2
    }
    // Altre funzioni del monitor
    function void Pn(...) {
        // Codice della funzione Pn
    }
}
```

Variabili di tipo condizione:

Per arricchire la definizione di monitor e consentire una maggiore flessibilità nella sincronizzazione, è necessario introdurre le variabili di condizione. Queste variabili permettono di eseguire due operazioni principali:

- **x.wait():** Mette in attesa il processo o il thread corrente sulla variabile di condizione 'x', forzandolo a lasciare il monitor.
- **x.notify():** Rende "pronto" un processo o un thread che era stato messo in attesa tramite 'x.wait()', se ce ne sono.

Concetto di monitor:

Il concetto di monitor in programmazione consente al programmatore di gestire la mutua esclusione nell'accesso a risorse specifiche e poter bloccare un thread in attesa del verificarsi di una certa condizione. In Java, il monitor è implementato a livello della classe 'Object', quindi ogni istanza di classe è anche un monitor, il cui funzionamento è attivato tramite l'uso di metodi synchronized. Per gestire l'accesso condizionato alle risorse, vengono utilizzati i metodi wait() e notify()/notifyAll(), che sono ereditati dalla classe Object e quindi disponibili in tutte le classi Java.

Tuttavia, mentre la gestione del lock intrinseco (usato con metodi synchronized) è trasparente, il programmatore è responsabile della gestione dei meccanismi di accesso condizionato alle risorse, utilizzando i metodi wait() e notify()/notifyAll().

Problema produttore consumatore:

Il problema del produttore/consumatore è un problema di sincronizzazione che si presenta nei sistemi concorrenti. In questo problema, ci sono due tipi di thread: il produttore e il consumatore, che operano su una risorsa condivisa, come un buffer. Il produttore è responsabile di produrre i dati e inserirli nel buffer, mentre il consumatore preleva i dati dal buffer e li utilizza.

Il problema si presenta quando il produttore tenta di inserire dati nel buffer quando è pieno, o quando il consumatore tenta di prelevare dati dal buffer quando è vuoto. Queste situazioni possono causare condizioni di attesa, deadlock o problemi di sovrascrittura dei dati.

Per risolvere il problema del produttore/consumatore in Java, si possono utilizzare varie tecniche di sincronizzazione, come i lock, i semafori, o i monitor. Ad esempio, si può utilizzare un monitor per garantire che il produttore aspetti quando il buffer è pieno e che il consumatore aspetti quando il buffer è vuoto. In alternativa, si possono utilizzare semafori per controllare l'accesso al buffer in modo che il produttore possa produrre solo se c'è spazio disponibile e il consumatore possa consumare solo se ci sono dati nel buffer.

Variabili atomiche e algoritmi non bloccanti:

L'atomicità può essere di due tipi:

- **Reale:** implica l'utilizzo di una sola istruzione di CPU per eseguire l'operazione, garantendo l'atomicità effettiva.
- **Virtuale:** il thread "crede" di avere accesso atomico alla variabile, concettualmente simile al concetto di monitor in Java con oggetti che implementano metodi 'synchronized'.

Operazione atomica:

Un'operazione è atomica quando tutte le sub-operazioni che la compongono verranno eseguite senza possibilità di interruzione da parte di un altro thread.

Problema della visibilità:

Durante l'accesso concorrente a una risorsa condivisa devono essere garantite due condizioni chiave:

- La mutua esclusione, la quale stabilisce che solo un thread può eseguire una sezione critica alla volta.
- La seconda è la visibilità, che assicura che le modifiche apportate da un thread ai dati condivisi siano visibili agli altri thread per mantenere la coerenza dei dati.

Tuttavia, i problemi di visibilità possono sorgere quando due thread operano su core diversi, poiché le variabili condivise vengono mantenute in cache per motivi di performance. Una soluzione a questo problema è l'uso di variabili **volatile**. I metodi e i blocchi sincronizzati sono altre opzioni che forniscono entrambe le proprietà desiderate, ma a scapito delle performance.

Problemi di locking:

Quando un thread non riesce ad acquisire un lock, viene sospeso, il che comporta un context switch che ha un costo. Durante l'attesa, un thread non può eseguire alcuna operazione, e questo può portare a situazioni di inversione di priorità, in cui un thread a bassa priorità blocca quelli con una priorità più alta. Questo fenomeno è noto come **priority inversion**. Il processo di locking è considerato pessimistico perché se la contesa è rara, l'acquisizione e l'esecuzione del lock possono aggiungere overhead senza una reale necessità. Inoltre, se le operazioni concorrenti sono limitate, come ad esempio gli aggiornamenti di una singola variabile, il costo dell'overhead del locking può superare il tempo necessario per l'accesso alla variabile stessa. È importante notare che un thread che possiede un lock potrebbe non essere in esecuzione.

Optimistic retrying:

La tecnica dell'Optimistic retrying consiste nel non utilizzare alcuna sincronizzazione durante le operazioni di lettura, ma solo durante le scritture, in particolare durante le operazioni di read-modify-write. Durante una scrittura, si procede come segue:

1. Si legge la variabile creando una copia locale.
2. Si aggiorna la copia.
3. Si tenta di scrivere nella variabile originale.

Questo approccio è implementato dagli strumenti `CompareAndSwap` e dal `compareAndSet`. Permane comunque, seppure in modo molto remoto, la possibilità di starvation, eliminando però i problemi legati al locking. Inoltre, nel peggiore dei casi, l'overhead non è mai superiore a quello dei lock.

Variabili atomiche in java:

Java dispone di 12 classi contenitori per implementare le variabili atomiche, su cui si possono effettuare operazioni di:

- `int get()` restituisce il valore corrente della variabile.
- `void set(int newValue)` aggiorna il valore corrente della variabile.
- `int getAndSet(int newValue)` aggiorna atomicamente il valore corrente della variabile e restituisce il valore contenuto precedentemente.
- `boolean compareAndSet(int expected, int update)` aggiorna atomicamente il valore della variabile `update` se il valore corrente della variabile è uguale a `expected`.

- `int getAndIncrement()` incrementa atomicamente il valore corrente della variabile e restituisce il valore contenuto precedentemente.

Oggetti complessi:

In Java, si può ricorrere a delle classi di aiuto, in cui salvare oggetti complessi, in particolare la `AtomicReference<T>` può essere usato per verificare l'aggiornamento dell'oggetto in questione usando la CAS.

Liveness:

Nella programmazione concorrente, due concetti fondamentali sono safety e liveness.

- Il concetto di safety riguarda il fatto che nessuno stato scorretto o incoerente sia raggiungibile dall'applicazione.
- Il concetto di liveness si riferisce alla capacità degli agenti di progredire correttamente nella loro elaborazione.

Tale proprietà che non è garantita dal fatto che le componenti concorrenti siano attive e accedano in maniera mutualmente esclusiva alle sezioni critiche di un oggetto (porzioni di codice ad accesso controllato).

I tre principali problemi di concorrenza sono:

- Deadlock.
- Starvation
- Livelock.

Deadlock:

Quando un processo ottiene l'accesso esclusivo a una risorsa, come avviene con la mutua esclusione, si possono verificare situazioni di stallo, note come deadlock, nella programmazione concorrente. Il deadlock si verifica quando ogni membro di un gruppo di agenti, come i thread, è in attesa che un altro membro rilasci un lock su una risorsa per poter procedere con la propria elaborazione.

Questa situazione crea un'attesa circolare che non si risolve mai, poiché nessuno dei thread può generare l'evento di sblocco, portando a un prolungamento infinito dell'attesa.

Condizioni necessarie per il deadlock:

Perché si verifichi un deadlock, devono verificarsi quattro condizioni fondamentali, spesso espresse in termini di risorse assegnate ai thread.

1. La prima è la mutua esclusione, che implica che solo un thread può utilizzare una risorsa alla volta.
2. La seconda è l'**hold and wait** o accumulo incrementale, in cui i thread in possesso di risorse possono richiederne altre senza rilasciare quelle già ottenute, creando un'attesa indefinita.
3. La terza condizione è l'assenza di preemption sulle risorse condivise, ovvero una risorsa può essere rilasciata solo volontariamente.
4. La quarta condizione è l'attesa circolare, in cui esiste una catena circolare di thread e richieste di accesso a risorse condivise, in cui ogni thread mantiene bloccate risorse richieste contemporaneamente da thread successivi nella catena.

Evitare i deadlock:

Per affrontare le situazioni di deadlock, esistono diversi approcci possibili:

- La **deadlock prevention** consiste nell'evitare che almeno una delle quattro condizioni necessarie (Mutual exclusion, Hold and wait, No preemption e Circular wait) per il deadlock si verifichi mai.
- Il **deadlock removal** consiste nel risolvere il lock una volta avvenuto senza prevederlo. Ad esempio, questo può essere fatto rendendo il possesso delle risorse interrompibile, permettendo allo scheduler di requisire e quindi liberare le risorse.

Starvation:

In programmazione concorrente, la starvation si verifica quando un agente è costantemente privato dell'accesso a una risorsa. Questa situazione può derivare da politiche dello scheduler che favoriscono certi processi a discapito di altri. Ad esempio, con uno scheduler a code con priorità statiche, molti processi ad alta priorità potrebbero impedire l'esecuzione di processi a priorità bassa. La starvation può anche essere causata da una gestione scorretta della mutua esclusione o da algoritmi di accesso alle risorse non ottimali.

I filosofi a cena:

Il problema presenta cinque filosofi che trascorrono la loro vita mangiando e pensando. Ogni filosofo ha davanti a sé un piatto e tra ciascuna coppia di piatti c'è una forchetta. Tuttavia, i filosofi condividono una sola forchetta tra due piatti, il che può portare a situazioni di deadlock se tutti i filosofi afferrano una forchetta contemporaneamente e nessuno può completare il proprio pasto.

Le possibili soluzioni del problema dei filosofi sono:

1. **Assegnazione asimmetrica delle forchette:** Consiste nell'assegnare una forchetta in modo asimmetrico ai filosofi, ad esempio permettendo a ciascun filosofo di afferrare prima la forchetta a sinistra e poi quella a destra. Questo impedisce la formazione di un ciclo di attesa tra i filosofi e previene il deadlock.
2. **Strategie di rilascio delle forchette:** I filosofi seguono regole specifiche per afferrare e rilasciare le forchette, evitando la competizione per un numero eccessivo di forchette e riducendo il rischio di deadlock. Ad esempio, potrebbero convenire di afferrare entrambe le forchette contemporaneamente solo se disponibili entrambe.
3. **Utilizzo di un arbitro:** Un arbitro o coordinatore gestisce l'accesso alle forchette, permettendo ai filosofi di richiedere e ottenere le forchette solo quando è sicuro farlo per evitare situazioni di deadlock.

Problema dei lettori e scrittori:

Il problema dei lettori/scrittori è un classico problema di sincronizzazione che coinvolge più processi che accedono a una risorsa condivisa. In questo scenario, la risorsa condivisa può essere letta da più processi contemporaneamente (lettori), ma può essere scritta solo da un processo alla volta (scrittore). L'obiettivo è garantire la coerenza dei dati durante l'accesso concorrente.

Le possibili soluzioni del problema dei lettori e scrittori sono:

1. **Soluzione senza preferenza:** In questa soluzione, viene garantito l'accesso prioritario agli scrittori. Quando uno scrittore richiede l'accesso, nessun altro lettore o scrittore può accedere alla risorsa fino a quando lo scrittore non ha finito di scrivere. Questo garantisce l'integrità dei dati durante la scrittura, ma potrebbe causare inattesa lentezza per i lettori.
2. **Soluzione con preferenza ai lettori:** In questa soluzione, viene garantito l'accesso prioritario ai lettori. I lettori possono accedere alla risorsa contemporaneamente, a meno che non ci sia

uno scrittore che richieda l'accesso. Quando uno scrittore richiede l'accesso, nessun altro lettore o scrittore può accedere alla risorsa fino a quando lo scrittore non ha finito di scrivere. Questo può portare a lunghe attese per gli scrittori.

Message oriented communication:

Browser:

Il browser è l'applicazione per il Web sul lato del client. Un web browser, detto User-Agent, è un programma che consente la navigazione nel Web da parte di un utente. La funzione primaria di un browser è quella di interpretare il codice con cui sono espresse le informazioni e visualizzarlo in formato di ipertesto.

Web page:

Una pagina web è essenzialmente un file digitale che risiede su un computer e può contenere una varietà di elementi come testo, immagini e musica. Questi file sono identificati da un URL unico che fornisce un modo per accedervi. Le pagine web sono create utilizzando il linguaggio HTML (HyperText Markup Language), che definisce la struttura e il contenuto della pagina, consentendo l'inserimento di testo, immagini, collegamenti e altri elementi. Il Web Server è l'applicazione responsabile della gestione di queste risorse sui computer e della loro distribuzione ai client. Questo significa che il Web Server controlla l'accesso e la disponibilità delle pagine web e di altri file su Internet. Una risorsa, come ad esempio uno studente o un conto corrente, può avere diverse rappresentazioni o profili che possono essere resi disponibili tramite il Web Server, garantendo la flessibilità nell'accesso e nella visualizzazione dei dati.

Ipertesto:

Il modello fondamentale del web è l'ipertesto, che consiste in testi o pagine interconnesse tramite link. Gli ipertesti permettono la lettura non sequenziale, poiché i lettori possono navigare attraverso i collegamenti seguendo percorsi definiti. Questi collegamenti creano una rete strutturata di informazioni, che possono essere organizzate in modo paritetico o gerarchico, ad esempio attraverso menu e sottosezioni.

URL:

Nell'ambito della rete, la comunicazione con un oggetto specifico avviene tramite un protocollo definito, che richiede cinque componenti principali:

- Il nome del protocollo.
- L'indirizzo dell'host di destinazione.
- La porta del processo (ovvero la controparte).
- Il percorso specifico dell'host.
- L'identificatore della risorsa desiderata.

Protocollo http:

Il protocollo http, HyperText Transfer Protocol, è un protocollo di livello applicativo utilizzato per la trasmissione di informazioni sul web, nella maggior parte dei casi per il trasferimento di pagine HTML.

- Comunicazione client-server: http opera su un modello client-server, dove il client inizia una connessione TCP creando una socket verso il server sulla porta 80. Quando il server accetta la connessione TCP dal client inizia lo scambio di messaggi http.

http è un protocollo di tipo **stateless**, ovvero non mantiene informazione sulle richieste precedenti del client.

Formato dei messaggi http:

I messaggi http possono essere di due tipi, request e response, che hanno però la stessa struttura:

- **Start-line:** obbligatorio, specifica il tipo di protocollo.
- **Header lines:** opzionali, sia nelle richieste che nelle risposte, le intestazioni trasportano metadati aggiuntivi. Queste informazioni includono dettagli sul tipo di dati, il tipo di codifica, le impostazioni di cache, i cookie e altre informazioni pertinenti alla richiesta o alla risposta.
- **Payload:** Opzionale per le richieste e le risposte, il corpo del messaggio trasporta i dati effettivi inviati dal client al server (nelle richieste) o dal server al client (nelle risposte).

Ogni riga del protocollo termina con il **Carriage-Return Line Feed**, un particolare tipo di sequenza di caratteri utilizzata per indicare la fine di una riga.

Esempio: messaggio http request

```
GET /somedir/page.html HTTP/1.1 -> request line
Host: www.someschool.edu
Connection: close -> richiede la chiusura della connessione
User-Agent: Mozilla/4.0 -> qualifica il richiedente
Accept: text/html, image/gif, image/jpeg
Accept-Language: fr
```

Metodi e applicazioni web:

- **GET:** il metodo GET viene descritto restituisce la rappresentazione di una risorsa, consentendo l'inclusione di parametri nella URL. È considerato safe, poiché non ha impatti sul server e la risposta può essere gestita tramite cache dal client. Questo metodo è comunemente utilizzato per ottenere dati come pagine HTML, immagini, XML o JSON.
- **POST:** con il metodo post i dati vengono inviati come parte del corpo del messaggio, dopo l'intestazione della richiesta. Il metodo POST non è "idempotente", il che significa che eseguire la stessa richiesta più volte può produrre risultati diversi o modificare lo stato del server. Il metodo POST è comunemente utilizzato per inviare dati da un modulo web a un server per l'elaborazione.
- **HEAD:** il metodo HEAD è un'opzione leggera e efficiente per ottenere informazioni sull'intestazione delle risorse HTTP senza scaricarne il contenuto completo. È ampiamente utilizzato per scopi di debugging, ottimizzazione delle prestazioni e controllo della cache.

Codici di stato response:

I codici di stato HTTP sono un insieme di numeri a tre cifre che vengono restituiti dal server web in risposta a una richiesta HTTP inviata dal client. Questi codici forniscono al client informazioni sullo stato della richiesta e dell'interazione con il server.

- **1xx - informative:** la richiesta è stata ricevuta e il server sta continuando il processo.
- **2xx - Success:** la richiesta è stata ricevuta, compresa e accettata con successo dal server.
- **3xx - Redirection:** il client deve effettuare ulteriori azioni per completare la richiesta.

- **4xx - Client Error:** c'è stato un errore nella richiesta del client e che il server non è in grado di completare la richiesta.
- **5xx - Server Error:** c'è stato un errore sul server e che il server non è in grado di completare la richiesta del client.

Esempi:

- 200 OK: successo, se richiesto l'oggetto è contenuto nel messaggio.
- 301 moved permantly: l'oggetto richiesto è stato spostato.
- 400 bad request: richiesta incomprensibile al server.
- 505 http version not supplied.

Tipi MIME:

I tipi MIME (Multipurpose Internet Mail Extensions) sono una convenzione utilizzata per identificare il tipo di contenuto di un file trasferito tramite Internet. I tipi MIME sono divisi in 5 sottotipi:

1. Text.
2. Image.
3. Audio.
4. Video.
5. Application.

Cookie:

I cookie sono utilizzati per memorizzare informazioni sul dispositivo dell'utente e per tracciare e personalizzare l'esperienza dell'utente su un sito web. Il cookie ha come obiettivo legare più richieste per associare un identificatore alla conversazione. Il Server invia un cookie che l'utente presenta in accessi successivi.

Comunicazione a flusso di messaggio:

Nel contesto dell'applicazione web, il processo di invio e ricezione dei messaggi avviene attraverso lo stream di byte che fluisce tra l'applicazione e il servizio di trasporto. L'applicazione invia i messaggi come stream di byte al servizio di trasporto, che a sua volta legge lo stream di byte e ricostruisce i messaggi per la consegna all'altro lato.

Per quanto riguarda i servizi di trasporto specifici come UDP e TCP, ciascuno ha il suo modo di gestire i dati.

Nel caso di UDP, il servizio riceve lo stream di byte e lo suddivide in segmenti. Successivamente, invia questi segmenti ai servizi di rete seguendo una politica specifica. Tuttavia, non vi è alcuna garanzia di consegna, e il riordino dei segmenti o la gestione delle perdite devono essere gestiti a livello di applicazione.

D'altra parte, TCP, un protocollo di trasporto affidabile, compie ulteriori passaggi per garantire l'affidabilità della trasmissione. Analogamente a UDP, scompone e invia i dati in segmenti, ma ogni segmento viene numerato per garantire il corretto riordinamento dei segmenti arrivati. Inoltre, TCP gestisce il controllo delle duplicazioni, scartando segmenti duplicati, e affronta le perdite ritrasmettendo i segmenti mancanti. Questi meccanismi assicurano una trasmissione affidabile dei dati attraverso la rete.

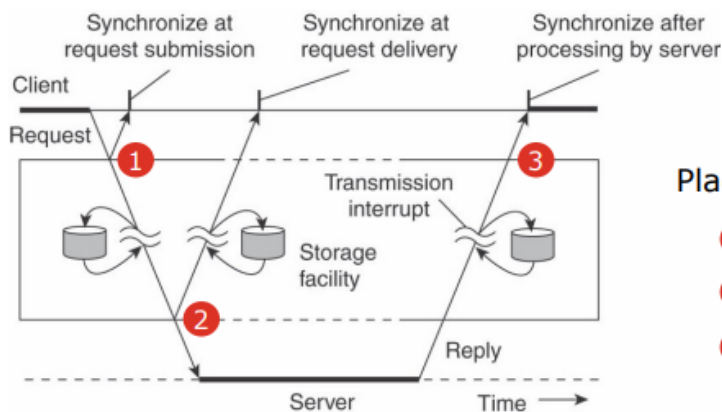
Tipi di comunicazione:

Una comunicazione può essere di diversi tipi:

- **Sincrona:** due o più interlocutori sono collegati contemporaneamente.
- **Asincrona:** non richiede il collegamento contemporaneo degli interlocutori alla rete di comunicazione.
- **Transiente:** la comunicazione transiente è una forma di comunicazione temporanea e senza stato che avviene senza la necessità di mantenere una connessione persistente tra mittente e destinatario.
- **Persistente:** la comunicazione persistente è una forma di comunicazione che avviene mantenendo una connessione costante o una sessione stabilita tra i partecipanti per un periodo di tempo prolungato.

Posizioni di sincronizzazione:

1. Alla somministrazione della richiesta.
2. Alla richiesta di invio
3. Dopo la richiesta di elaborazione.

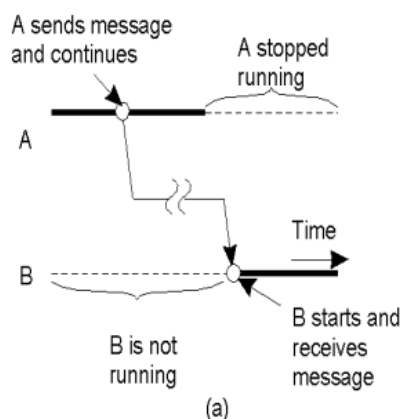


Places for synchronization

- ① At request submission
- ② At request delivery
- ③ After request processing

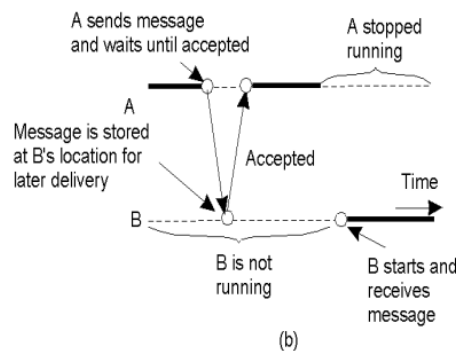
Comunicazione persistente asincrona:

L'entità A invia un messaggio e continua il suo processo anche se l'entità B non è in esecuzione. L'entità B può avviare il suo processo e ricevere il messaggio in un momento successivo. Questo è utile in scenari in cui non è necessario che entrambe le entità siano attive contemporaneamente per la comunicazione.



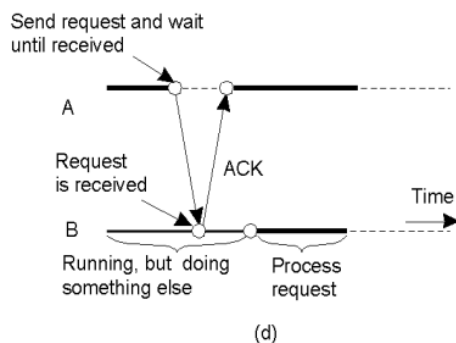
Comunicazione persistente sincrona:

L'entità A invia un messaggio ma attende fino a quando non viene accettato da B, che non è in esecuzione al momento. Una volta che B avvia il suo processo e accetta il messaggio, A continua il suo processo. Questo è utile in scenari in cui A deve assicurarsi che il messaggio sia stato ricevuto da B prima di continuare.



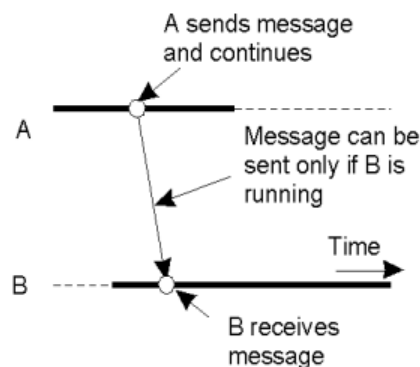
Comunicazione sincrona transitoria basata sulla ricevuta:

La comunicazione sincrona transitoria basata sulla ricevuta è un tipo di comunicazione in cui un mittente invia un messaggio a un destinatario e poi si blocca fino a quando il messaggio non viene ricevuto.



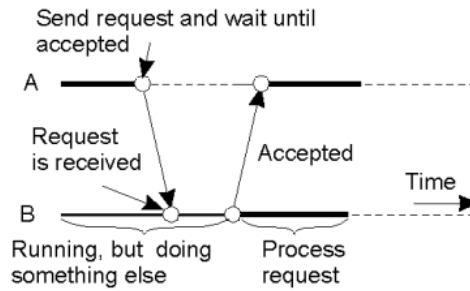
Comunicazione transitoria asincrona:

L'entità A invia un messaggio e continua, il messaggio può essere inviato solo se B è in esecuzione, se B è in esecuzione il messaggio viene ricevuto altrimenti viene perso e A continua per la sua strada.



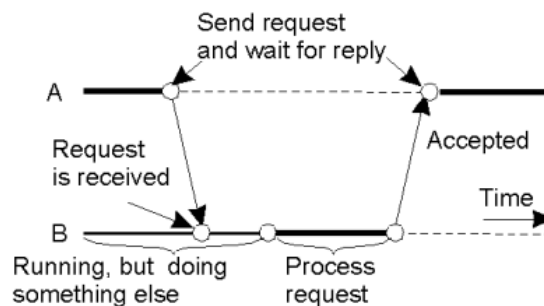
Comunicazione sincrona transitoria basata sulla consegna:

La comunicazione sincrona transitoria basata sulla consegna è un tipo di comunicazione in cui il mittente rimane bloccato fino a quando il destinatario non ha ricevuto il messaggio. Questo scenario è tipico dei sistemi di messaggistica istantanea.



Comunicazione sincrona transitoria basata sulla risposta:

La comunicazione sincrona transitoria basata sulla consegna è un tipo di comunicazione in cui il mittente attende una risposta dal destinatario prima di procedere. Questo scenario si presenta durante le chiamate telefoniche o le videochiamate.

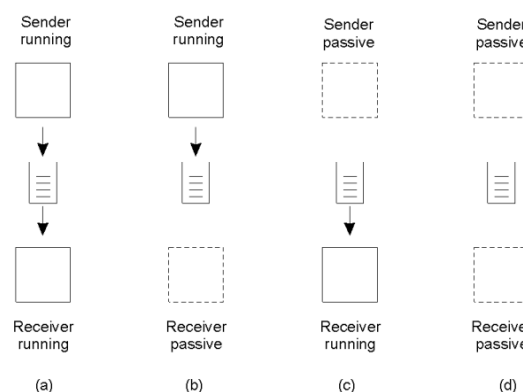


Message-Queuing Model:

I sistemi di accodamento dei messaggi forniscono un ampio supporto per la comunicazione asincrona persistente. L'essenza di questi sistemi è che offrono una capacità di archiviazione a medio termine per i messaggi, senza richiedere che il mittente o il destinatario siano attivi durante la trasmissione del messaggio.

L'idea di base dietro a un sistema di accodamento dei messaggi è che le applicazioni comunicano inserendo messaggi in code specifiche. Questi messaggi vengono inoltrati attraverso una serie di server di comunicazione e vengono infine consegnati alla destinazione, anche se era inattiva quando il messaggio è stato inviato.

Ogni applicazione ha una coda privata per ricevere messaggi da altre applicazioni. Una coda è leggibile solo dall'applicazione a cui è associata, ma può essere condivisa tra più applicazioni. Il mittente ha solo la garanzia che il suo messaggio sarà inserito nella coda del destinatario, senza alcuna certezza su quando o se il messaggio sarà effettivamente letto, poiché ciò dipende dal comportamento del destinatario. Questo tipo di comunicazione, liberamente accoppiata nel tempo, non richiede che mittente e destinatario siano attivi contemporaneamente.



La Figura illustra quattro scenari di comunicazione:

- La figura A mostra che sia il mittente che il destinatario sono attivi durante l'intera trasmissione del messaggio.
- La figura B mostra che solo il mittente è attivo e può inviare messaggi, mentre il destinatario è in uno stato passivo e non può ricevere messaggi.
- La figura C mostra un mittente passivo e un destinatario attivo. In questo scenario, il destinatario può leggere i messaggi inviati, indipendentemente dallo stato del mittente.
- La figura D mostra sia il mittente che il destinatario sono passivi, ma il sistema continua a memorizzare e potenzialmente a trasmettere messaggi.

Dal punto di vista del middleware, l'aspetto cruciale è l'indirizzamento corretto dei messaggi, che viene realizzato fornendo un nome unico per la coda di destinazione a livello di sistema. Sebbene possa esserci un limite alla dimensione del messaggio in alcuni casi, il sistema sottostante può gestire la frammentazione e l'assemblaggio di messaggi di grandi dimensioni in modo trasparente per le applicazioni.

In un sistema di accodamento dei messaggi, un mittente utilizza la primitiva "put" per inviare un messaggio al sistema sottostante per essere aggiunto a una coda specificata. Questa è una chiamata non bloccante. D'altra parte, la primitiva "get" è una chiamata bloccante che permette a un processo autorizzato di rimuovere il messaggio più vecchio in attesa nella coda specificata, bloccando il processo solo se la coda è vuota. Esistono variazioni di questa chiamata che consentono di cercare un messaggio specifico nella coda, ad esempio, utilizzando una priorità o un pattern corrispondente.

La primitiva "poll" è una variante non bloccante. Se la coda è vuota, o se un messaggio specifico non può essere trovato, il processo chiamante continua semplicemente. Infine, molti sistemi di accodamento permettono a un processo di installare un gestore come funzione di callback, che viene automaticamente invocata ogni volta che un messaggio viene inserito nella coda. Queste callback possono anche essere utilizzate per avviare automaticamente un processo che preleverà i messaggi dalla coda se nessun processo è attualmente in esecuzione.

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages and remove the first. Never block.
Notify	Install a handler to be called when a message is put into the specified queue.

Message broker:

Un'area cruciale di utilizzo dei sistemi di messaggistica è l'integrazione di applicazioni, sia nuove che esistenti, in un'unica piattaforma informativa distribuita. Questo richiede che le applicazioni siano in grado di comprendere i messaggi che ricevono, il che implica che mittente e destinatario debbano aderire allo stesso formato di messaggio. Tuttavia, l'adozione di un formato comune spesso non è praticabile, poiché i sistemi di messaggistica operano a livelli di astrazione diversi. In alternativa, i messaggi possono essere convertiti mediante l'uso di un "message broker", un nodo speciale all'interno della rete di code. Questo broker svolge il ruolo di gateway a livello di applicazione, garantendo che i messaggi in ingresso siano trasformati nel formato comprensibile per l'applicazione

destinataria. In pratica, ciò significa che un message broker può riformattare i messaggi in arrivo per adattarli alle specifiche richieste della destinazione.

Publish and subscribe pattern:

Il pattern Publish and Subscribe è un modello di comunicazione utilizzato nei sistemi di messaggistica per la distribuzione di messaggi tra più componenti del sistema in modo asincrono. Questo modello coinvolge tre parti principali: i publisher, i subscribers e il message broker.

- **Publisher:** Sono le entità che generano e inviano i messaggi all'interno del sistema. I publisher non sono consapevoli dei destinatari specifici dei loro messaggi. Invece, inviano i messaggi a un topic (argomento) specifico senza conoscere i sottoscrittori interessati. Possono essere applicazioni, servizi o altri componenti del sistema.
- **Subscriber:** Sono le entità interessate a ricevere i messaggi su argomenti specifici. I subscriber si registrano presso il message broker per ricevere i messaggi relativi a determinati argomenti. Una volta registrati, il message broker instrada i messaggi pertinenti ai sottoscrittori corrispondenti quando vengono pubblicati dai publisher.

Il pattern permette una comunicazione flessibile e decentralizzata tra i componenti di un sistema. I publisher inviano i messaggi senza sapere chi li riceverà, mentre i subscriber ricevono solo i messaggi relativi agli argomenti a cui si sono sottoscritti. Il message broker facilita questo scambio di messaggi, garantendo che i messaggi vengano consegnati ai destinatari appropriati. Questo modello è ampiamente utilizzato in applicazioni distribuite, sistemi di eventi in tempo reale, e nell'implementazione di microservizi.

WEB-app SERVLET

Applicazione web:

Un'applicazione web è un'applicazione distribuita, ovvero accessibile via web per mezzo di un network, come ad esempio internet all'interno di un sistema informatico offrendo determinati servizi all'utente client.

Tecnologie server side:

La computazione nel contesto delle applicazioni web avviene principalmente sul server. Questo processo può avvenire attraverso l'esecuzione di programmi compilati o script interpretati. L'application server svolge un ruolo cruciale fornendo un ambiente per gestire in modo automatizzato il ciclo di vita di tali programmi. Nel caso dei programmi compilati, il web server si limita a invocare un eseguibile in risposta alle richieste dei client. Per quanto riguarda gli script, il web server incorpora un motore in grado di interpretare il linguaggio di scripting utilizzato. Sebbene l'esecuzione degli script sia generalmente più lenta rispetto ai programmi compilati, la loro scrittura è più agevole e rapida.

Architettura di un'applicazione web compilata:

L'architettura di un'applicazione compilata richiede la presenza di una Common gateway Interface, protocollo che permette al server di attivare un programma, passargli le richieste e i parametri provenienti dal client e recuperare la risposta elaborata.

Architettura di un'applicazione web interpretata:

In questo caso, il protocollo CGI viene gestito dall'interprete del linguaggio utilizzato e non dalle applicazioni. Rispetto all'approccio precedente, ha una serie di vantaggi quali una maggiore velocità di sviluppo, flessibilità durante l'esecuzione e facilità di debugging.

Client side HTML:

Nel client side, l'elaborazione è trasparente e viene presentato solo il risultato, in formato HTML. Il risultato viene elaborato da un browser, che può effettuare diversi tipi di richieste.

Richieste basate su link:

Un link in un documento HTML può essere usato per puntare ad una risorsa remota. Il server invierà richieste del tipo GET/Servlet/GetHTTP HTTP/1.1.a.

Richieste per mezzo di Form:

Viene utilizzato il parametro action di un form per puntare ad una risorsa remota, ovvero ad un'applicazione. Il server invierà richieste del tipo GET/Servlet/GetHTTP HTTP/1.1.a.

Richieste con dati:

Un form può essere usato anche per mandare dati al server. In questo caso, la richiesta inviata dal browser sarà più complicata, del tipo:

```
POST /SlideServlet/PostHTTPServlet HTTP/1.1
Content-Length: 11
Content-Type: application/x-www-form-urlencoded
param=value
```

La richiesta, in questo caso, è stata effettuata tramite il metodo POST; è possibile utilizzare anche il metodo GET, che accoda i parametri all'URL della richiesta effettuata, elaborando una richiesta del tipo:

```
<form action="http://localhost:8080/Servlet/PostHTTP" method="POST">
  What is your favorite pet?<br><br>
  <input type="radio" name="animal" value="dog">Dog<br>
  <input type="radio" name="animal" value="cat">Cat<br>
  <input type="radio" name="animal" value="bird">Bird<br><br>
  <input type="submit" value="Submit">
  <input type="reset">
</form>
```

Server side Java Servlet:

Le servlet sono piccole applicazioni Java residenti sul server, sono gestite automaticamente da un container, devono implementare un'interfaccia prestabilita che definisce un set di metodi. Offrono vantaggi in termini di semplicità e standardizzazione, ma presentano anche rigidità nel modello. Il container controlla le servlet, attivandole o disattivandole in base alle richieste dei client, grazie alla conoscenza dell'interfaccia implementata dalle servlet. Le servlet possono anche interagire con altre servlet.

Stateless vs Stateful:

Nell'ambito della comunicazione HTTP, esistono due approcci:

1. Stateless.

2. Stateful.

HTTP è intrinsecamente stateless, il che significa che non conserva informazioni tra una richiesta e l'altra, né identifica i client. Tuttavia, mantenere lo stato della conversazione è possibile attraverso l'uso di meccanismi esterni come i cookies, che memorizzano informazioni sul client. Le sessioni di lavoro possono essere gestite attraverso l'uso di oggetti come `HTTPSession`, che sono gestiti automaticamente dal container o dall'engine e possono essere accessibili alle servlet per immagazzinare informazioni. In questo modo, l'applicazione può mantenere lo stato della conversazione anche in un protocollo intrinsecamente stateless come HTTP.

Interfaccia servlet:

Ogni servlet implementa l'interfaccia `jakarta.servlet.Servlet`, con cinque metodi:

<code>void init(ServletConfig config)</code> inizializza la servlet, viene invocato dopo la creazione della stessa
<code>void destroy()</code> chiamata quando la servlet termina (es: per chiudere un file o una connessione con un database)
<code>void service(ServletRequest request, ServletResponse response)</code> invocato per gestire le richieste dei client
<code>ServletConfig getServletConfig()</code> restituisce i parametri di inizializzazione e il <code>ServletContext</code> che dà accesso all'ambiente
<code>String getServletInfo()</code> restituisce informazioni tipo autore e versione

Classe `HTTPServlet`:

Implementa `service()` in modo da invocare i metodi per servire le richieste dal web, Implementa anche diversi metodi del tipo `doX`, per effettuare `get`, `post`...

Oltre a queste, sono presenti anche metodi per gestire le richieste e le risposte:

- `String getParameter(String name)` restituisce il valore del query parameter dato il nome, un valore singolo.
- `Enumeration getParameterNames()` restituisce l'elenco dei nomi degli argomenti.
- `String[] getParametersValues(String name)` restituisce i valori dell'argomento `name`, un valore multiplo.

Ciclo di vita di una servlet:

Quando una servlet viene chiamata per la prima volta, il container servlet la crea e la condivide tra tutti i client. Ogni richiesta genera un thread dedicato che esegue il metodo appropriato `doXXX`. Durante questa fase, il container invoca il metodo `init()` per effettuare inizializzazioni specifiche. La servlet viene distrutta dal container quando non ci sono più thread in esecuzione su di essa o quando scade un timeout predefinito. La terminazione avviene attraverso l'invocazione del metodo `destroy()`. Tuttavia, alla scadenza del timeout, potrebbero ancora esserci dei thread in esecuzione in `service()`. Pertanto, è necessario progettare il metodo `destroy()` in modo che notifichi lo shutdown e attenda il completamento del metodo `service()`. Inoltre, i metodi devono implementare tecniche per verificare periodicamente se è in corso uno shutdown e comportarsi di conseguenza.

Servlet e thread:

Le servlet in Java sono multithreading, il che significa che possono gestire più richieste contemporaneamente utilizzando thread separati per ciascuna richiesta. Tuttavia, esse sono progettate per essere stateless, quindi ogni richiesta è trattata indipendentemente dalle altre e i thread non condividono lo stato. È importante gestire correttamente le risorse condivise tra i thread per evitare

problemi di concorrenza, utilizzando strutture dati sincronizzate o meccanismi di sincronizzazione come i lock.

Server side JSP:

JSP (Java Server Pages) è una tecnologia per la creazione di applicazioni web che definisce l'interazione tra un server e un insieme di pagine che mostrano informazioni agli utenti. Le pagine sono composte sia da tag tradizionali (come HTML, XML, WML) sia da tag applicativi che gestiscono la generazione del contenuto sul server. Rispetto alle servlet, JSP semplifica la separazione tra la logica dell'applicazione e la presentazione dei dati.

È simile alla tecnologia Microsoft Active Server Page (ASP), ma presenta alcune differenze chiave. Ad esempio, mentre una Java Server Page richiama un programma Java eseguito sul server web, un Active Server Page contiene uno script VBScript o JScript. Questa distinzione rende JSP una scelta preferita per chi desidera sfruttare la potenza di Java nell'ambito dello sviluppo web.

Java server pages:

Le Java Server Pages (JSP) consentono di separare la parte dinamica delle pagine dal template HTML statico. Il codice JSP è incluso in tag delimitati da “<%” e “%>”. Una Java Server Page funziona convertendo dinamicamente le pagine JSP in servlet Java ogni qualvolta vengono richieste dal client.

Elementi di una JSP:

- **Template text:** sono le parti statiche di una pagina html.
- **Commenti:** <%- - questo è un commento - -%>
- **Direttive:** <%@ direttiva di compilazione %> le direttive forniscono istruzioni al container JSP su come elaborare la pagina stessa.
- **Azioni:** <jsp:XXX attributes> body </jsp:XXX> le azioni permettono di eseguire codice Java all'interno della pagina.
- **Elementi di scripting:** consentono di incorporare codice Java direttamente all'interno delle pagine JSP.

Direttive:

- **Direttive di pagina:** forniscono informazioni specifiche sulla pagina stessa. Alcuni attributi includono:
 - **Import:** specifica i pacchetti Java che devono essere importati nella pagina.
 - **Session:** indica se la pagina utilizza oggetti di sessione.
- **Direttive di inclusione:** includono il contenuto di un altro file all'interno della pagina JSP durante la compilazione.
- **Direttive di taglib:** consentono di dichiarare e utilizzare librerie di tag personalizzate all'interno della pagina JSP.

Azioni:

- **Azioni di forward:** reindirizzano dinamicamente la richiesta del client verso un'altra risorsa (come un'altra pagina JSP o una servlet) durante il processo di rendering.
- **Azioni di inclusione:** vengono utilizzate per includere dinamicamente il contenuto di un'altra risorsa all'interno della pagina corrente durante il processo di rendering.

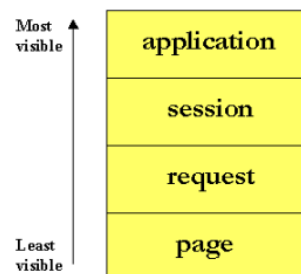
- **Azioni useBeans:** sono utilizzate nelle JSP per creare o recuperare un oggetto JavaBeans e memorizzarlo all'interno dello scope specificato ad esempio, la richiesta, la sessione o l'applicazione.

Elementi di scripting:

- **Declaration:** le dichiarazioni iniziano con `<%!` e terminano con `%>`, sono utilizzate per dichiarare variabili e metodi statici che possono essere utilizzati in tutta la pagina JSP.
- **Expression:** le espressioni iniziano e terminano con `<%=` e `%>`, consentono di valutare espressioni Java e includere il risultato direttamente nell'output HTML della pagina.
- **Scriptlets:** gli scriptlets iniziano e terminano con `<%` e `%>`. Consentono di inserire blocchi di codice Java all'interno della pagina JSP, che verranno eseguiti quando la pagina viene elaborata dal server.

Oggetti:

Gli oggetti sono entità Java che possono essere memorizzate in vari scope per essere accessibili da diverse parti dell'applicazione web.



- **Applicazione:** gli oggetti memorizzati nello scope dell'applicazione sono accessibili da tutte le pagine JSP all'interno dell'applicazione web.
- **Sessione:** gli oggetti memorizzati nello scope della sessione sono accessibili da tutte le pagine JSP all'interno della stessa sessione utente.
- **Richiesta:** gli oggetti memorizzati nello scope della richiesta sono accessibili da tutte le pagine JSP all'interno della stessa richiesta http.
- **Pagina:** gli oggetti memorizzati nello scope della pagina sono accessibili solo dalla pagina JSP in cui sono stati creati o recuperati.

JavaBean:

Un JavaBean è una classe Java che segue determinate convenzioni di progettazione per rappresentare un oggetto con uno stato ben definito e un set di metodi per accedere a tale stato. Le caratteristiche di un javaBean sono le seguenti:

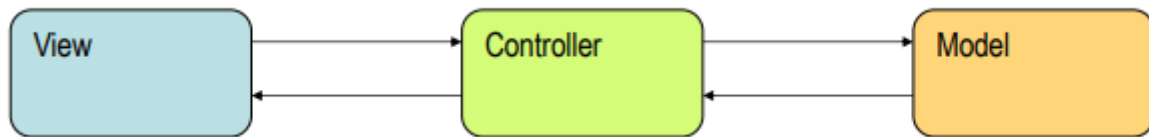
- Costruttore senza parametri.
- Attributi privati.
- Metodi getter e setter.

Pattern MVC (model view control):

Il pattern Model View Controller ha lo scopo di separare:

- **Model:** Contiene i dati e i metodi per manipolarli.
- **View:** Gestisce la presentazione, ovvero l'interfaccia utente.

- **Controller:** Coordinatore dell'interazione tra l'interfaccia (le azioni degli utenti) e i dati.



Vantaggi e svantaggi del pattern MVC:

Vantaggi del pattern MVC:

- Chiara separazione tra logica di business e logica di presentazione.
- Possibilità di cambiare la View senza modificare il Controller e viceversa.
- Aggiunta di funzionalità alla View senza appesantire il Controller.
- Flessibilità nel definire e selezionare la View in base all'interazione, ai dispositivi, allo stato dei dati o alle preferenze dell'utente.
- Chiara separazione tra logica di business e modello dei dati.
- Possibilità di definire mapping diversi tra azioni dell'utente e funzioni del Model.
- Responsabilità ben definite per ogni componente.
- Possibilità di sviluppare in parallelo e mantenere e far evolvere ogni componente in modo indipendente.

Svantaggi del pattern MVC:

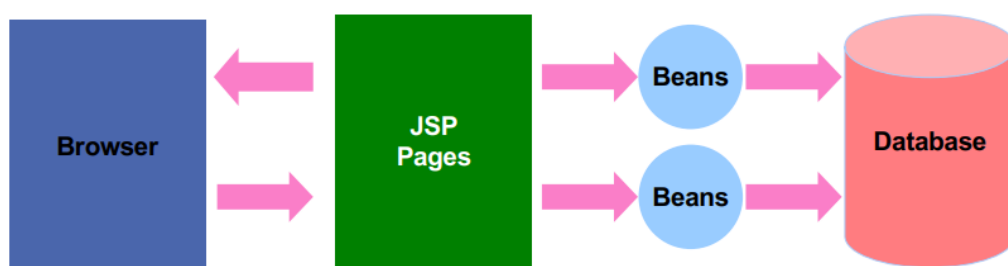
- Aumento della complessità dovuto alla gestione della concorrenza (essendo un sistema distribuito).
- Inefficienza nel passaggio dei dati alla View (un elemento aggiuntivo tra cliente e controller).

Pattern model 1:

Lo scopo del pattern model 1 è separare i dati, la logica di business e la visualizzazione. In breve, il processo avviene in questo modo:

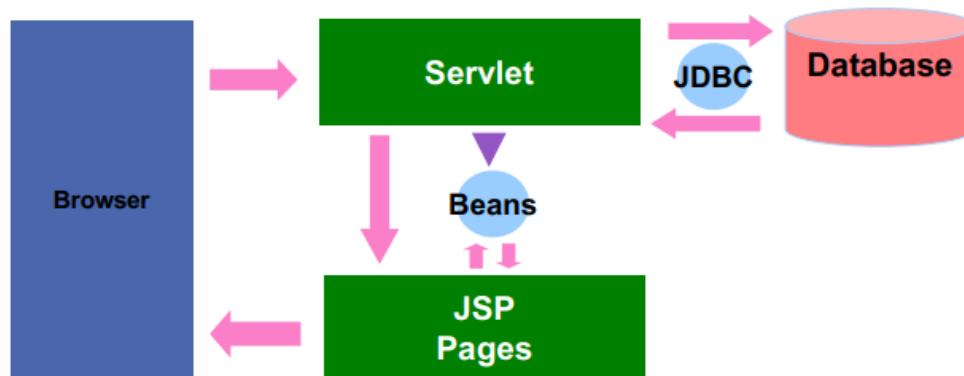
- Il browser invia una richiesta per la pagina JSP.
- La JSP accede ai Java Bean e invoca la logica di business.
- I Java Bean si connettono al database e ottengono/salvano i dati.
- La risposta, generata dalla JSP, viene inviata al browser.

Uno svantaggio del pattern Model 1 consiste nel controllo decentralizzato della navigazione, poiché ogni pagina contiene la logica per determinare la pagina successiva. Inoltre, per evitare che le pagine JSP diventino troppo piene di codice Java, potrebbe essere necessario creare molti tag ad hoc per suddividere e organizzare la logica di business e di presentazione.



Pattern model 2:

Nel pattern model 2 la richiesta viene inviata ad una Java Servlet che genera i dati dinamici richiesti dall'utente e li mette a disposizione della pagine JSP come JavaBeans. La servlet richiama quindi una pagina JSP che legge i dati dai beans e organizza la presentazione in HTML che invia all'utente.



Service computing:

IOT:

L'Internet of Things è un sistema di oggetti fisici che possono essere scoperti, monitorati, controllati o interagire con altri dispositivi elettronici, che comunicano attraverso varie interfacce di rete e possono essere collegati ad internet.

Architettura orientata ai servizi (SOA):

L'Architettura Orientata ai Servizi, è uno stile architettonico che privilegia l'uso di elementi discreti e riutilizzabili chiamati servizi anziché un design monolitico per lo sviluppo di applicazioni. Ogni servizio fornisce specifiche funzionalità agli utenti, che possono essere altri servizi, e l'accesso avviene tramite la rete, incluso Internet. Le sue funzionalità includono l'accesso tramite protocolli di rete standard, la composizione con altri servizi per creare soluzioni complesse, la risposta alle esigenze aziendali e dei clienti, e il mantenimento di un livello di qualità del servizio definito.

Servizio web:

Un servizio web è un'applicazione software che utilizza le tecnologie web, in particolare l'HTTP, per fornire funzionalità. Ogni servizio web è identificato da un URI unico, che serve come suo indirizzo su Internet. Le interfacce di un servizio web sono definite in modo tale da poter essere descritte e scoperte, permettendo così ad altre applicazioni di interagire con esso. Infine, un servizio web supporta interazioni dirette con altri software attraverso l'uso di messaggi e protocolli basati su Internet.

Accordo sul livello di servizio (service level agreement):

Un SLA è un contratto tra un fornitore di servizi (il provider) e un utente che assicura che le funzionalità siano fornite correttamente. Questo contratto garantisce anche le proprietà non funzionali del servizio. Le caratteristiche non funzionali garantite dal servizio sono definite nell'SLA. Inoltre, un SLA comprende diversi Service Level Objectives (SLO), che definiscono la qualità del servizio da garantire attraverso metriche specifiche. Queste metriche possono includere, ad esempio, la disponibilità del servizio, il tempo di risposta, la capacità di gestire un certo numero di richieste, ecc. In questo modo, l'SLA fornisce un quadro chiaro e misurabile delle aspettative di servizio tra il fornitore e l'utente.

Servizio di composizione:

La composizione di servizi consiste nell'interconnessione di un insieme di servizi, che possono essere utilizzati come nuovi servizi in altre composizioni. Due servizi sono considerati interconnessi se almeno uno dei due richiede la funzionalità esposta (conosciuta anche come endpoint o API) dell'altro. Perché la composizione dei servizi abbia successo, è necessaria la compatibilità tra i servizi. In altre parole, devono "parlare la stessa lingua", sia a livello sintattico che semantico. Questo significa che devono essere in grado di comprendere e interpretare correttamente i dati che scambiano.

Infine, un fornitore di servizi deve presentare un'interfaccia pubblicata. Questa interfaccia definisce le funzionalità che il servizio offre e come possono essere utilizzate da altre applicazioni o servizi. In questo modo, altri servizi possono facilmente integrarsi e interagire con il servizio.

Business process:

Un business process è un insieme di attività correlate, o workflow, che vengono eseguite da persone e applicazioni per ottenere un risultato ben definito, come un servizio o un prodotto. I Business Process software sono creati dalla composizione, o integrazione, di vari servizi.

Orchestrazione vs coreografia:

- **Orchestrazione:** Questo termine descrive come i servizi interagiscono tra loro, compresa la logica di business e l'ordine di esecuzione delle interazioni dal punto di vista e sotto il controllo di un singolo attore (servizio). Ad esempio, si ha orchestrazione quando un servizio utilizza le funzionalità di altri servizi (nel giusto ordine) per implementare la propria logica di business. L'orchestrazione richiede un controllo attivo, il che la rende più complicata, ma anche più facile da monitorare.
- **Coreografia:** Questo termine descrive la sequenza di interazioni tra più parti coinvolte nel processo dal punto di vista di tutte le parti. Definisce lo stato condiviso delle interazioni tra le entità. In una coreografia, ogni servizio è responsabile di portare a termine il proprio compito, invocando anche altri servizi se necessario. Di solito, la coreografia viene implementata utilizzando eventi.

Enterprise service bus:

L'architettura SOA ha semplificato l'integrazione tra i diversi componenti di un sistema informativo, definendo a priori i messaggi scambiabili. Questo ha portato alla definizione dell' Enterprise Service Bus (ESB), un sistema di comunicazione progettato per supportare l'interazione e la comunicazione tra i componenti di un sistema informativo. L'ESB è un esempio di approccio coreografico e di solito implementa il modello publish/subscribe. Le caratteristiche principali di un ESB includono l'instradamento dei messaggi tra applicazioni e servizi, la trasformazione del messaggio, la comunicazione sicura e un'architettura estensibile. In sintesi, l'architettura SOA e l'ESB hanno reso più semplice e flessibile l'integrazione di vari componenti in un sistema informativo.

Soap services (Simple Object Access Protocol):

SOAP è un protocollo di comunicazione che utilizza il linguaggio XML per codificare i messaggi, rendendoli facilmente leggibili sia dagli umani che dalle macchine. È indipendente dal sistema di trasporto, il che significa che può essere utilizzato con qualsiasi protocollo. Funziona su un modello di richiesta e risposta, dove un programma invia una richiesta SOAP a un altro programma e riceve una risposta. SOAP è estensibile attraverso l'uso di namespace XML, il che significa che può essere personalizzato per soddisfare le esigenze specifiche di un'applicazione. Un namespace XML è un identificatore che permette di distinguere elementi e attributi con lo stesso nome. Inoltre, è neutrale

rispetto al linguaggio di programmazione, quindi può essere utilizzato con qualsiasi linguaggio. Infine, è spesso utilizzato nei servizi web orientati ai documenti, dove il messaggio stesso contiene tutte le informazioni necessarie per il destinatario per capire e processare la richiesta.

Componenti di un messaggio SOAP:

Un messaggio SOAP è composto principalmente da tre componenti:

- **Envelope:** contiene tutti gli elementi del messaggio.
- **Header:** è elemento opzionale che contiene metadati per il messaggio, come la crittografia utilizzata. Può includere informazioni come dettagli di autenticazione, informazioni sulla transazione.
- **Body:** è l'elemento obbligatorio che contiene i dati effettivi del messaggio. Include le informazioni che si desidera inviare o ricevere.

SOAP con http:

Sebbene SOAP possa utilizzare diversi protocolli di trasporto, HTTP è quello più comunemente utilizzato. Le richieste HTTP sono formate da un metodo, come POST o GET, seguito dall'URL richiesto e dalla versione del protocollo, ad esempio HTTP 1.1. Le risposte seguono la semantica di HTTP per fornire il codice di stato della risposta; ad esempio, un codice di stato del tipo 2xx indica che la richiesta è stata ricevuta, compresa e accettata.

Esistono due modelli principali per lo scambio di messaggi SOAP tramite HTTP:

- Modello **SOAP request-response**, in cui il metodo POST viene utilizzato per inviare i messaggi SOAP nel corpo delle richieste/risposte HTTP.
- Modello **SOAP response**, in cui il metodo GET viene utilizzato nelle richieste HTTP per ottenere il messaggio SOAP nel corpo della risposta.

WSDL (Web Service Description Language):

Il Web Services Description Language è un linguaggio basato su XML utilizzato per definire come e dove accedere ad un servizio. Serve a descrivere i servizi web, i messaggi e le modalità di invocazione. Un file WSDL contiene una descrizione dell'interfaccia di un servizio e permette di descrivere quattro dati principali per un servizio:

- Le informazioni sull'interfaccia, che descrivono tutte le operazioni pubblicamente disponibili di un servizio.
- Le dichiarazioni di tipi di dati per tutti i messaggi. I tipi complessi possono essere dichiarati (utilizzando SOAP) e utilizzati.
- Le informazioni di binding sul protocollo di trasporto, come HTTP, SMTP, UDP.
- Le informazioni sull'indirizzo per la localizzazione del servizio, solitamente sotto forma di URI.

REST services:

Principi di REST (REpresentational State Transfer):

REST è un'architettura software per i sistemi distribuiti è un modello progettuale che definisce un insieme di vincoli e principi per creare servizi web efficienti, scalabili e interoperabili.

Caratteristiche di REST:

Una risorsa è una qualsiasi informazione che può essere nominata: documenti, immagini, servizi, collezioni. Ogni risorsa in un servizio REST ha un identificatore, un path parameter utilizzato per identificare una specifica risorsa o un gruppo di risorse.

Le risorse possono essere manipolate utilizzando metodi HTTP standard come:

- GET;
- POST;
- PUT;
- DELETE;
- PATCH.

L'architettura client-server prevede una chiara separazione dei ruoli tra i client e i server, il che porta a una serie di vantaggi in termini di scalabilità, gestione e manutenzione dei sistemi, il client invia richieste ad un sistema server per recuperare o manipolare dati.

La manipolazione di risorse attraverso rappresentazioni si riferisce al modo in cui un client interagisce con una risorsa sul server. Una rappresentazione in questo contesto è una forma concreta di una risorsa in un determinato momento.

Inoltre, le risorse in REST sono stateless, il che significa che ogni richiesta da parte del client deve contenere tutte le informazioni necessarie per comprendere e processare la richiesta. Il server non conserva alcuno stato tra le richieste.

Caching:

Nell'architettura REST, la cacheability richiede un sistema a livelli. Deve esistere almeno un livello di cache dove le risposte possono essere memorizzate per un uso futuro. Questa cache memorizza l'ultima rappresentazione di una risorsa, permettendo un recupero più veloce dei dati senza dover fare una nuova richiesta al server. Tuttavia, quando lo stato di una risorsa cambia sul server (ad esempio, attraverso operazioni PUT o DELETE), la rappresentazione della risorsa nella cache diventa obsoleta e deve essere invalidata. Questo assicura che i client ricevano sempre i dati più aggiornati, mantenendo al contempo le prestazioni e l'efficienza attraverso l'uso efficace della cache.

La cacheability è una delle caratteristiche fondamentali che contribuisce a migliorare le prestazioni del sistema riducendo il carico sui server e diminuendo i tempi di risposta per i client. Le parti cacheabili in REST includono le seguenti componenti:

- **Risposte GET:** le risposte alle richieste GET sono cacheabili poiché il metodo GET è idempotente e non modifica lo stato del server.

Costruzione di applicazioni REST

Si seguono quattro passi consecutivi:

1. Trovare tutti i nomi bisogna definire URI descrittive, opache e persistenti, che si possano evolvere solo attraverso il versionamento. Bisogna evitare query, gerarchie, uso di verbi e preferire i sostantivi.
2. Definire i formati, evitando di creare rappresentazioni personalizzate.
3. Scegliere le operazioni possibili, che spesso sono le 4 operazioni GET, POST, PUT, DELETE che sono sufficienti, ma esistono comunque altri metodi http.
4. Evidenziare i codici di stato eccezionali.

Hypermedia Control:

L'Hypermedia Control è un concetto chiave nell'architettura REST. Si tratta di un'estensione dell'idea di ipertesto, che è fondamentale per il funzionamento del web. In un contesto REST, l'Hypermedia Control si riferisce all'uso di collegamenti ipertestuali nelle risposte API per guidare le azioni successive che un client può intraprendere.

Questo concetto è spesso espresso con l'acronimo **HATEOAS**, che sta per “**Hypermedia As The Engine Of Application State**”. In pratica, significa che un'API REST fornisce collegamenti ipertestuali con ogni risposta che collegano ad altre risorse correlate. Ad esempio, una risposta API potrebbe includere non solo i dati richiesti, ma anche collegamenti a risorse correlate, come dettagli aggiuntivi o azioni correlate.

L'Hypermedia Control ha diversi vantaggi. Permette ai client di interagire con l'API in modo più dinamico e flessibile, poiché possono scoprire azioni disponibili e risorse correlate attraverso i collegamenti forniti piuttosto che dover conoscere in anticipo tutte le possibili richieste API.

RIA (Rich Internet Application):

Le pagine web vengono sempre ricaricate e non vengono mai aggiornate, l'utente aspetta che tutta la pagina venga caricata e tutto viene eseguito con la restrizione di una sola richiesta e risposta. A causa di queste limitazioni viene introdotto l'Ajax, una tecnica di sviluppo per creare applicazioni web interattive che utilizzano un modello asincrono.

Usi di Ajax:

Alcuni esempi di utilizzo di Ajax:

- Validazione dei dati in tempo reale nei form;
- Auto-completamento dei form;
- In base a un evento del client, una pagina HTML può recuperare informazioni più dettagliate sui dati senza aggiornare la pagina;
- Controlli UI più sofisticati.

Componenti di Ajax:

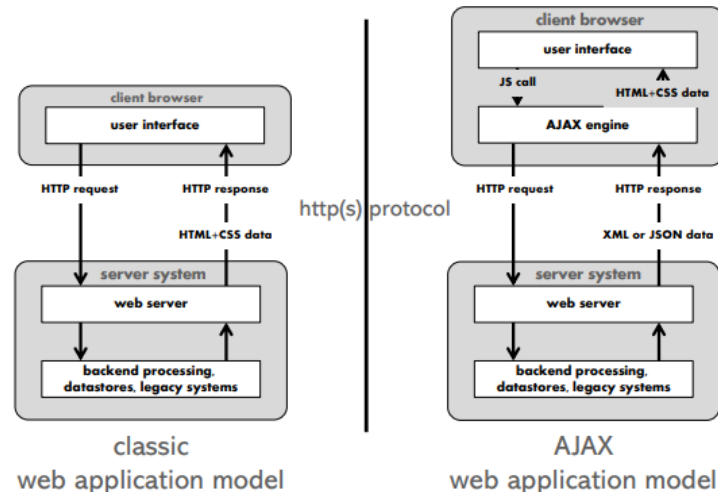
- HTML e CSS per la rappresentazione delle informazioni.
- DOM (Document Object Model) per l'organizzazione gerarchica della pagina web.
- XML e XSLT che sono modelli per rispettivamente rappresentare e trasformare i dati.
- XMLHttpRequest effettua richieste asincrone al server web.

JavaScript:

JavaScript è un linguaggio di scripting a oggetti, non tipizzato, che viene interpretato da un engine ed eseguito dal browser. JavaScript consente di rendere le pagine HTML dinamiche, inserendo programmi che modificano il comportamento e le visualizzazioni. È importante perché può effettuare richieste HTTP al server in modo trasparente per l'utente e rendere asincrona la comunicazione tra browser e web server. JavaScript può richiedere dati in formato testo puro e XML, anche se attualmente il formato più diffuso è JSON.

Architettura classica VS architettura dinamica:

- Nel modello classico, l'interazione tra il browser del client e il server avviene attraverso una sequenza di richieste e risposte HTTP. In questo modello, ogni interazione richiede il caricamento completo della pagina web, il che può risultare in un'esperienza utente meno fluida e più lenta.
- Nel modello AJAX, la comunicazione tra il client e il server è più efficiente e permette un'interazione più dinamica e reattiva.



Problemi delle RIA:

Ci possono essere alcuni problemi alle tecniche RIA, conseguenti soprattutto dalle caratteristiche asincrone e dall'elaborazione lato client:

- C'è interferenza con la funzionalità del tasto "indietro" del browser, che potrebbe presentare pagine diverse da quelle precedentemente viste.
- Alcune sezioni della pagina potrebbero cambiare inaspettatamente.
- Aumenta il carico da parte del browser, bisogna infatti usare una bilancia: capire quanto è conveniente usare il codice, e quanto i dati.
- È difficile il debug, perchè gli errori sono presenti sia sul client sia sul server.
- La sorgente è completamente visibile, quindi si introducono problemi di sicurezza e affidabilità.

Trasmissione dei dati in RIA:

La trasmissione dei dati tra server e applicazioni RIA è un aspetto critico e può avvenire tramite diverse tecnologie come SOAP, XML-RPC, JSON e AMF. La scelta del formato dei dati influenza la struttura dell'applicazione e le performance, poiché ogni formato richiede tempo per predisporre i dati lato server, trasferirli, fare il parsing lato client e fare il rendering dell'interfaccia.

Ajax (Asynchronous JavaScript and XML):

Ajax è un insieme di tecniche di sviluppo web che consentono alle applicazioni web di funzionare in modo asincrono, elaborando qualsiasi richiesta al server in background. Grazie ad AJAX, una pagina web può scambiare dati con il server in background senza dover essere completamente ricaricata.

Struttura di una pagina:

La struttura di una pagina si attiene al formato HTML; quindi, si ha una struttura HTML che include i contenuti da presentare, l'interfaccia utente e le modalità di interazione input e output.

Dato che una pagina HTML viene elaborata sequenzialmente, gli script Ajax, inclusi nei tag `<script></script>`, devono essere posti nella head, oppure in coda al resto del codice HTML. Infatti, gli elementi HTML devono essere già caricati e disponibili, affinché Ajax possa manipolarli.

Data output:

JavaScript non ha funzioni integrate per stampare o visualizzare dati, ma può "visualizzare" dati in vari modi:

- Scrivendo in un elemento HTML tramite innerHTML, ad esempio, sostituendo il contenuto di un elemento con un identificatore specifico.
- Scrivendo all'interno di un alert box utilizzando window.alert(), che mostra il contenuto indicato in una nuova finestra.
- Scrivendo nella console del browser con console.log(), utile per i messaggi di debugging.
- Scrivendo all'interno dell'output HTML usando document.write(), che sostituisce l'intera pagina.

Function invocation in JavaScript:

L'invocazione di una funzione in JavaScript è il processo di eseguire il codice definito all'interno di una funzione. Ecco un approfondimento sui vari modi in cui una funzione può essere invocata:

1. Avviene un certo evento.
2. Javascript invoca la funzione.
3. Invocata automaticamente (self-invoked).

JavaScript Events:

Gli eventi in JavaScript sono azioni o occorrenze che avvengono nel browser e possono essere rilevate tramite JavaScript. Per aggiungere un evento ad un elemento, si utilizza la sintassi: `<element event="JavaScriptCode()">`

Paradigma ad eventi:

Il paradigma ad eventi, noto anche come programmazione basata sugli eventi, è un modello di progettazione del software in cui il flusso del programma è determinato da eventi come le azioni dell'utente. Gli eventi sono azioni o occorrenze rilevate dal software che possono essere gestite dal programma. Possono essere generati dall'utente, dal sistema o da altri programmi. Un gestore di eventi è una funzione di callback che gestisce un particolare evento. Quando si verifica un evento, il gestore di eventi corrispondente viene eseguito. Il loop di eventi è un ciclo di controllo del programma che ascolta gli eventi e li distribuisce ai relativi gestori di eventi.

AddEventListener:

AddEventListener è un metodo in JavaScript utilizzato per ascoltare gli eventi su un elemento HTML e associare loro delle funzioni (event handlers) che verranno eseguite quando l'evento si verifica.

element.**addEventListener**(event, function, useCapture);

- **element:** È l'elemento HTML a cui si desidera aggiungere l'ascoltatore di eventi.
- **event:** È una stringa che rappresenta il tipo di evento da ascoltare, come "click", "mouseover", "keydown", ecc.

- **function:** È la funzione che verrà eseguita quando l'evento si verifica.
- **useCapture (opzionale):** È un valore booleano che specifica se utilizzare il meccanismo di capturing o bubbling per gestire l'evento.

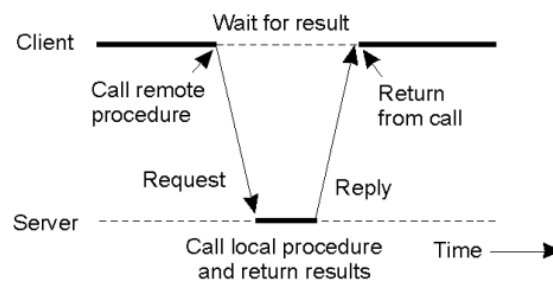
Remote Procedure Call:

La Remote Procedure Call è una tecnologia che consente a un programma per computer di eseguire una procedura (sottoprogramma) su un altro spazio di indirizzi (di solito su un altro computer su una rete condivisa) come se fosse una procedura locale, cioè come se fosse chiamata all'interno del proprio programma.

Il modello RPC maschera il modello client-server, la differenza principale sta nella gestione delle informazioni scambiate. Nel modello RPC si utilizzano i parametri.

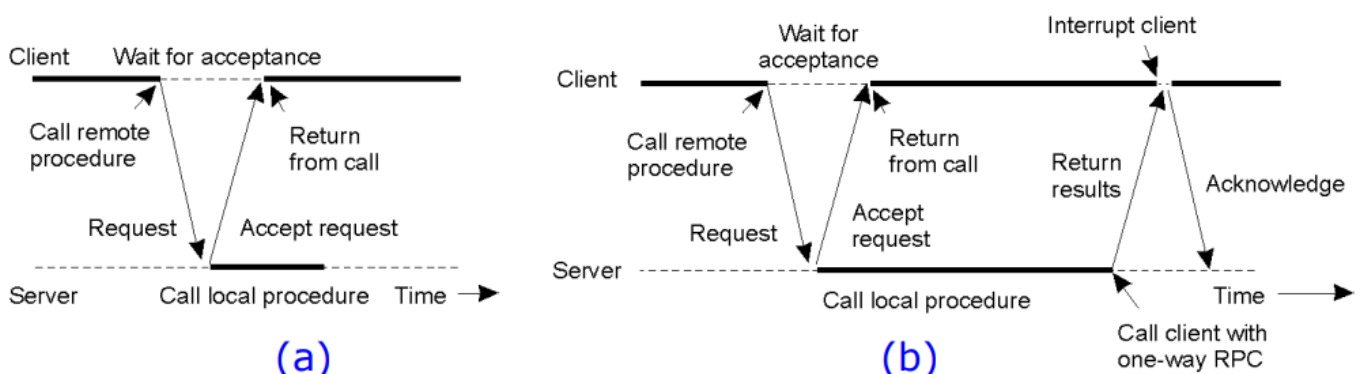
RPC sincrona:

L'interazione tra un client e un server utilizzando RPC asincrona con una risposta posticipata (callback) può essere utile in scenari dove il server impiega un tempo significativo per elaborare una richiesta. In questo modello, il client invia una richiesta asincrona al server e continua con altre operazioni. Quando il server completa l'elaborazione, invia una seconda richiesta asincrona (callback) al client con il risultato. Questo approccio aumenta l'efficienza e la reattività del client.



RPC asincrona:

Il modello di Remote Procedure Call (RPC) asincrona è un'estensione del modello RPC standard che permette a un client di inviare una richiesta al server senza aspettare che il server completi l'elaborazione e ritorni il risultato.



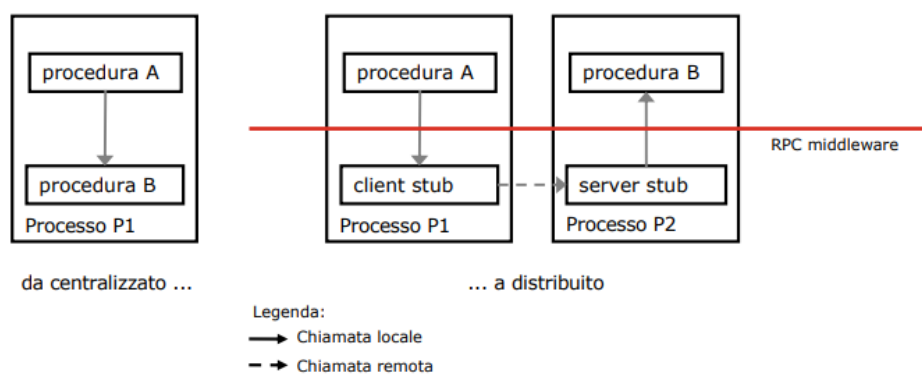
- Interazione tra client e server utilizzando una RPC asincrona, senza risposta.
- Interazione tra client e server utilizzando una RPC asincrona, con risposta posticipata utilizzando una seconda RPC asincrona.

Architettura RPC:

L'architettura RPC è un modello di comunicazione che consente a un programma di eseguire una procedura su un computer remoto come se fosse locale. Questo modello nasconde la complessità della comunicazione di rete, rendendo le chiamate remote trasparenti al programmatore.

Le componenti principali dell'architettura RPC sono:

- **Client:** L'entità che desidera chiamare una funzione remota. Il client invia una richiesta al server per eseguire una specifica procedura e attende la risposta.
- **Stub del Client:** Una funzione proxy sul lato client che impacchetta i parametri della richiesta (marshalling) e invia la richiesta al server. Nasconde i dettagli della rete dal codice dell'applicazione client.
- **Sistema di Comunicazione:** Il meccanismo che trasmette le richieste e le risposte tra il client e il server. Utilizza protocolli di rete come TCP o UDP.
- **Stub del Server:** Una funzione proxy sul lato server che riceve la richiesta, la scompone (unmarshalling), chiama la funzione reale sul server e impacchetta la risposta per inviarla indietro al client.
- **Server:** L'entità che riceve la richiesta dal client esegue la funzione richiesta e restituisce il risultato al client.



Passaggio di valori dei parametri:

Quando si implementa una computazione remota tramite RPC, la computazione stessa segue una serie di step:

- Il client richiama la procedura remota
- Lo stub costruisce il messaggio
- Il messaggio viene inviato, attraverso la rete, al server
- Il sistema operativo del server gestisce il messaggio e lo trasferisce allo stub del server
- Lo stub spacchetta il messaggio
- Lo stub gestisce localmente la richiesta

I parametri localmente vengono passati attraverso i vari step tramite dei puntatori; invece, quando si arriva alla call remota si replica lo stack nel processo remoto: i dati vengono impacchettati, spediti, spacchettati per creare copie locali. La regola che guida tutto è che si deve simulare l'ambiente locale della controparte. Questa operazione si chiama marshalling e unmarshalling.

Distributed Objects:

Tipi di oggetti:

- Oggetti incapsulano i dati e definiscono le modalità di accesso e le operazioni che si effettuano su di loro.
- Oggetti a compile-time definiti attraverso interfacce e classi.
- Oggetti a run-time accessibili attraverso adapters, detti anche wrappers.
- Oggetti persistenti e transienti.
- Riferimenti a degli oggetti remoti.

Puntatore:

Un puntatore è una variabile che contiene un indirizzo di memoria, può essere inizializzato in qualsiasi momento offrendo una maggiore flessibilità nel controllo della memoria. Durante il suo ciclo di vita, un puntatore può puntare a molti oggetti diversi, rendendolo versatile per varie operazioni di programmazione. È possibile creare un array di puntatori, il che consente di gestire facilmente un gruppo di indirizzi di memoria. Per accedere al valore a cui punta un puntatore, è necessario utilizzare un operatore esplicito (*), noto come operatore di dereferenziazione. Inoltre, un puntatore può essere non inizializzato, il che significa che non deve necessariamente riferirsi a un oggetto valido. Infine, un puntatore, anche se definito come costante, può avere un valore NULL, indicando che non punta a nessun indirizzo di memoria valido.

Riferimento:

Un riferimento, indicato con &, si presenta come un alias per una variabile esistente e deve essere inizializzato al momento della sua definizione. Questo significa che un riferimento non può essere riassegnato per riferirsi a un altro oggetto dopo la sua inizializzazione. Questa caratteristica garantisce che il riferimento sia sempre legato a un particolare oggetto. A differenza dei puntatori, non è possibile creare un array di riferimenti, poiché ogni riferimento nell'array dovrebbe essere inizializzato al momento della creazione. Per accedere al valore di un riferimento, non è necessario utilizzare alcun operatore di dereferenziazione, rendendo l'uso dei riferimenti più semplice e diretto. Un riferimento valido deve sempre riferirsi a un oggetto esistente, e non può mai essere NULL, assicurando che non ci siano riferimenti a indirizzi di memoria non validi.

Java RMI (Java Remote Method Invocation)

Java RMI (Remote Method Invocation) è un middleware che estende l'approccio orientato agli oggetti al contesto distribuito, consentendo l'ereditarietà e supportando l'invocazione di metodi tra oggetti su macchine virtuali separate. Le interfacce sono definite in Java anziché in un linguaggio di definizione dell'interfaccia generico. Viene consentito il passaggio e il ritorno di oggetti Java, mentre le classi sono caricate dinamicamente, sfruttando la portabilità del bytecode e della macchina virtuale, rendendo il sistema più sicuro ed efficiente. Java RMI fornisce diversi servizi, tra cui il caricamento e il controllo delle classi tramite un class loader e un security manager, la gestione di oggetti remoti, replicati e persistenti, l'attivazione automatica degli oggetti, il supporto al multi-threading e la garbage collection degli oggetti remoti utilizzando un meccanismo di conteggio dei riferimenti.

Java environment:

La class loader è incaricata di caricare le classi Java. L'attività di caricamento dinamica avviene quando un oggetto RMI viene trasferito.

Invocazioni:

Java RMI è simile a RPC per la gestione dei parametri per valore, ma consente anche il passaggio di parametri per riferimento. Definisce stub specifici per ogni oggetto, a differenza di RPC che usa stub generici.

Passaggio di parametri:

In Java, i tipi primitivi e gli oggetti serializzabili non remoti vengono trasferiti per valore, ossia viene creata una copia dei dati. Tuttavia, i riferimenti agli oggetti remoti vengono passati per riferimento per consentire invocazioni remote implementando le interfacce `Remote` e `Serializable`.

Serializzazione:

La serializzazione in Java converte lo stato di un oggetto in uno stream di byte, permettendo la memorizzazione e ricostruzione degli oggetti per il trasferimento via rete o per la persistenza. Grazie al meccanismo di loading dinamico di Java, vengono trasmesse solo le informazioni essenziali sullo stato, mentre la descrizione della classe può essere caricata separatamente.

La serializzazione utilizza il metodo `writeObject(Object obj)` della classe `Object`, che attraversa tutti i riferimenti contenuti nell'oggetto per creare una rappresentazione completa del grafo degli oggetti.

Identificazione degli oggetti:

In Java RMI, i nomi assegnati dall'utente vengono convertiti in riferimenti operativi utilizzando un servizio di directory come `rmiregistry`, che deve essere disponibile su un host e una porta noti, convenzionalmente la porta 1099. Questo processo consente ai client di localizzare e interagire con gli oggetti remoti tramite nomi intuitivi anziché indirizzi di rete complessi. Ecco come funziona in dettaglio.

Classe naming:

La classe `Naming` di Java RMI fornisce metodi statici per registrare, trovare, disattivare e ottenere un elenco di oggetti remoti nel registro RMI. Il registro RMI è un servizio di directory che associa nomi stringa a oggetti remoti, permettendo ai client di localizzare e accedere a questi oggetti.

- `Lookup`: restituisce un riferimento, uno stub, all'oggetto associato al nome specificato
- `Bind`: collega il nome specificato all'oggetto remoto
- `List`: restituisce i nomi, in formato URL, degli oggetti del registry
- `Unbind`: distrugge il collegamento al nome specificato
- `Rebind`: collega il nome specificato all'oggetto remoto cancellando i collegamenti esistenti. Si potrebbe pensare di usare un'altra volta il metodo `bind`, ma potrebbero esserci problemi di concorrenza e nomi duplicati; pertanto, si adotta questo metodo per sostituire un oggetto "aggiornandolo"

Architettura RMI:

Nell'architettura RMI, il server pubblica il riferimento e il nome dell'oggetto remoto nel registro RMI utilizzando il metodo `bind()`. Questo processo permette al registro di associare un nome univoco all'oggetto remoto. Successivamente, il client ottiene il riferimento all'oggetto remoto invocando il metodo `lookup()` del registro RMI, specificando il nome dell'oggetto. Una volta ottenuto il riferimento, il client può accedere all'oggetto remoto.

Problemi fondamentali di RMI:

- **Naming:** utilizza il nome dell'host e il nome dato all'oggetto per identificarlo in modo univoco.
- **Access Point:** il registro RMI converte la coppia <host, nome dell'oggetto> in un riferimento. I nomi locali forniti semplificano la scrittura dei programmi.
- **Formato dei Dati:** utilizza oggetti Java e supporta la serializzazione locale degli oggetti o il riferimento agli oggetti remoti.
- **Semantica dei Dati:** utilizza i tipi Java, includendo classi e metodi, per garantire la coerenza semantica durante la comunicazione tra client e server.

Creazione di un oggetto remoto:

Java definisce un'interfaccia per implementare oggetti remoti, la `java.RMI`. Per creare una classe remota serve:

1. Definire l'interfaccia della classe remota.
2. Implementare la nuova interfaccia.
3. Implementare un server che crei e registri l'oggetto al Registry.

Il server crea un oggetto locale che realizza il servizio desiderato, e lo registra presso l'RMI registry sullo stesso host con un nome pubblico.

Programmazione JavaScript:

Variabili e costanti:

Le variabili sono dichiarate **var** e valgono per tutto il programma, (**global scope**) o solo all'interno di funzioni (**function scope**). Le variabili dichiarate **let** sono visibili solo all'interno del blocco in cui sono dichiarate (**block scope**). Infine, le **const** permettono di definire variabili **let** con un valore costante, cioè che non può essere modificato. In realtà, sono contenitori costanti, ma il loro valore può cambiare. Le variabili semplici sono imm modificabili, mentre i valori delle variabili composte possono cambiare.

Forme sintattiche per le funzioni:

Definizione di funzioni in ES5:

```
function f(x, y) {  
    return x * y;  
}  
  
var f = function(x, y) {  
    return x * y;  
}
```

Definizione di funzioni in ES6: ES6 introduce una nuova sintassi per definire le funzioni: arrow function

```
const f = (x, y) => x * y;
```

La keyword `return` e le parentesi graffe `{ }` possono essere omesse solo quando la funzione ha una sola istruzione:

```
const f = (x, y) => {return x*y};
```

Il tipo string in JavaScript:

In JavaScript, il tipo stringa rappresenta una sequenza di caratteri testuali. Le stringhe possono essere definite utilizzando singoli o doppi apici, e possono contenere qualsiasi carattere Unicode, inclusi lettere, numeri, spazi, simboli e caratteri di escape.

To find the length of a string, use the built-in `length` property:

```
let sln = carName1.length;
```

The `+` operator can be used to add (concatenate) strings:

```
let txt1 = "John";  
let txt2 = 'Doe';  
let txt3 = txt1 + " " + txt2;
```

Metodi del tipo string:

- The `indexOf()` method returns the index of the first occurrence of a specified text in a string.
- The `lastIndexOf()` method returns the index of the last occurrence of a specified text in a string.

Both `indexOf()`, and `lastIndexOf()` return -1 if the text is not found

- The `startsWith()` [`endsWith()`] method returns true if a string begins [ends] with a specified value, otherwise false (case sensitive).

An optional second parameter indicates a position to start from [end to].

- The `slice()` method extracts a part of a string and returns the extracted part in a new string. It takes 1 or 2 parameters: the start position, and the end position (excluded) [optional]

Metodi per la manipolazione degli array:

Gli array, dichiarati come `const`, vengono manipolati con diverse funzioni standard:

- `push()` inserisce un elemento in ultima posizione.
- `pop()` rimuove l'ultimo elemento.
- `shift()` rimuove il primo elemento ed effettua lo shift degli altri.
- `unshift()` inserisce un elemento in prima posizione.
- `splice()` inserisce un elemento ad un indice dato.

Classi in JavaScript:

La versione di JavaScript ES6 ha introdotto le classi: servono per definire dei prototipi di oggetti per dichiarare più entità oggetto dello stesso tipo. La creazione avviene attraverso l'operazione `new` seguito dal nome della classe e da eventuali parametri. Per convenzione, i nomi delle classi iniziano con la lettera maiuscola.

Iterazioni su variabili multivalore:

In JavaScript, le variabili multivalore possono essere iterate utilizzando diversi metodi, a seconda del tipo di dati. Le strutture dati più comuni per variabili multivalore sono gli array e gli oggetti. Ecco come è possibile iterare su di essi:

L'istruzione `for/of` considera tutti i valori di una struttura iterativa (array, stringhe, ...)

```
const numbers = [1, 2, 3, 4, 5, 6];
let sum = 0;
for (let x of numbers) {
  sum = sum + x;
}
// sum vale 21
```

L'istruzione `for/in` considera tutte le proprietà di un oggetto:

```
const person = {fname:"John", lname:"Doe", age:25};
let str = "";
for (let x in person) {
  str += x + ": " + person[x] + "<br>";
}
// str: "fname: John<br>lname: Doe<br>age: 25<br>"
```

Elementi di programmazione aggiuntivi:

Ci sono alcuni elementi di programmazione caratteristici e con funzionalità che aiutano la programmazione, come:

- **ForEach:** itera gli elementi di un array.
- **Map:** mappa gli elementi di un array, data una funzione.
- **Filter:** crea un nuovo array con gli elementi di un array che passano un certo test filtro, programmato come una funzione booleana.
- **Reduce:** produce un singolo valore da un array.

Oggetti JSON:

La JavaScript Object Notation è un formato leggero per lo scambio di dati, facile da leggere e scrivere per l'uomo e per la macchina. È costruito su due strutture universali:

- **Coppia chiave valore:** una collezione di coppie che associano ad ogni chiave un valore.
- **Una lista ordinata di valori:**
 - **Oggetto:** una lista disordinata di coppie chiave valore.
 - **Array:** una collezione ordinata di valori.
 - **Valore:** un valore può essere una stringa, un numero, un booleano, un oggetto oppure un array.
 - **Stringa:** una stringa è una sequenza di zero o più caratteri unicode racchiusi tra i doppi apici.

Node.js:

Node.js è una piattaforma realizzata su V8, il motore Javascript di Chrome, che permette di realizzare applicazioni web veloci e scalabili. V8 è un esecutore open source, molto efficiente che occupa pochissime risorse, usa un modello di I/O non bloccante e ad eventi per creare un framework modulare.

Il modello asincrono utilizzato prevede di non aspettare che l'effetto di una funzione sia completato prima di eseguire la prossima. Creo una funzione di callback che completi l'effetto desiderato mentre proseguo l'esecuzione delle istruzioni successive.

Modello ad eventi:

Il modello ad eventi è sviluppato su un singolo thread, è un loop che esegue e verifica periodicamente l'avvenire di un certo evento. Tuttavia, ciò comporta il blocco della UI a causa delle lunghe esecuzioni.

Modello di esecuzione:

L'esecuzione dei programmi in Node si basa su un single event loop, che preleva un evento da una singola coda e lo serve eseguendo le operazioni previste.

Il loop si trova al livello logico, le operazioni sono eseguite logicamente in sequenza. A livello fisico ogni operazione è eseguita da un thread autonomo che permette l'esecuzione parallela e concorrente. Questo modello si basa sull'esecuzione di operazioni stateless e disaccoppia la programmazione dall'esecuzione favorendo la scalabilità.

Realizzare una Web app in un Application Server

Framework express:

Il modulo express.js è un framework web che fornisce gli strumenti per realizzare un server che può ospitare sia risorse statiche, sia applicazioni che generano rappresentazioni dinamiche.

Rappresentazione dell'informazione:

I linguaggi di marcatura sono strumenti utilizzati per annotare documenti elettronici, consentendo di separare e gestire contenuto, presentazione e navigazione. La marcatura consente di rendere esplicita un'interpretazione di un testo, inserendo codice all'interno di un documento elettronico. Esistono due tipi di marcatura:

- **Marcatura procedurale:** descrive come processare il documento (es. PostScript, PDF, RTF, formato MS Word);
- **Marcatura descrittiva:** descrive la struttura logica del documento (es. HTML, SGML, XML).

XML – eXtensible Markup Language:

XML si è affermato come lo standard per lo scambio di informazioni semi-strutturate. I documenti XML sono caratterizzati da una marcatura descrittiva, che descrive la struttura logica del documento, a differenza della marcatura procedurale. Questa marcatura offre flessibilità, permettendo che le etichette (tag) cambino in base all'applicazione. La struttura della marcatura ammessa è specificata dal tipo di documento, definito tramite la Document Type Declaration (DTD), che è parte integrante dello standard XML.

Documento XML:

Un documento XML è una struttura di dati che segue specifiche regole di formattazione e organizzazione per essere "**ben formato**". Questo significa che deve contenere uno o più elementi con un solo elemento radice che racchiude tutti gli altri e assicurare che ogni entità referenziata sia

ben formata. Un documento XML può anche essere "valido" se rispetta ulteriori vincoli di validazione, definiti tramite una DTD o uno schema XML, che specifica la struttura e il tipo di dati ammessi.

Document type declaration:

La Document Type Declaration (DTD) in XML è una parte del documento che definisce la struttura e le regole per gli elementi e gli attributi contenuti nel documento stesso. Serve a specificare quali elementi possono apparire, come possono essere nidificati e quali attributi possono avere, assicurando che il documento sia organizzato in modo coerente e possa essere validato. La DTD può essere inclusa direttamente nel documento XML o essere referenziata esternamente.

Limitazioni della DTD:

Le DTD (Document Type Declarations) descrivono solo la "grammatica" di un file XML, senza specificare la struttura dettagliata e i tipi dei dati. Questo approccio presenta alcune limitazioni evidenti:

- Non è possibile esprimere che un elemento "length" debba contenere un numero non negativo;
- L'elemento "unit" dovrebbe essere usato solo quando è presente "amount";
- L'elemento "comment" dovrebbe poter apparire ovunque (flessibilità dello schema);
- Non esiste subtyping o ereditarietà;
- Non ci sono chiavi composite.

XML schema:

XML Schema è un linguaggio di schema più sofisticato rispetto ai DTD, supporta la tipizzazione dei valori (ad esempio, interi, stringhe) e consente di impostare vincoli su valori minimi e massimi. Inoltre, permette la definizione di tipi complessi personalizzati e offre numerose altre funzionalità, come vincoli di unicità, chiavi esterne e ereditarietà.

HTML (HyperText Markup Language):

HTML è un linguaggio di markup per dare struttura ai contenuti.

CSS (Cascading Style Sheets):

È un linguaggio per dare uno stile (di presentazione/visuale) ai contenuti. Le media query in CSS sono selettori che valutano le capacità del dispositivo di accesso, come schermi, stampanti o sistemi text-to-speech. Permettono di controllare parametri come larghezza e altezza del device o della finestra, l'orientamento dello schermo e la risoluzione. Ad esempio, se la pagina è più larga di 480 pixel, è possibile applicare specifici stili agli elementi con id "leftsidebar" e "main" per adattare la visualizzazione.

Il termine "cascading" in CSS si riferisce alla gestione dei potenziali conflitti tra diversi stylesheet. Questi possono provenire dall'autore della pagina, dal browser o dal lettore che personalizza la propria esperienza. Poiché esistono più fonti di stile, è necessario un algoritmo che decida quale stile applicare a un elemento. La priorità degli stili è determinata da tre fattori: importanza (con il flag 'important'), specificità (id > class > tag) e ordine nel sorgente (l'ultimo stile dichiarato prevale).

DOM (Document Object Model):

È una interfaccia neutrale rispetto al linguaggio di programmazione e alla piattaforma utilizzata per consentire ai programmi l'accesso e la modifica dinamica di contenuto, struttura e stile di un documento. DOM è una API definita dal W3C e implementata da ogni browser moderno per l'accesso e la gestione dinamica di documenti XML e HTML.

Semantica dei dati:

Il web semantico:

Il web semantico è un'estensione del web che promuove formati di dati comuni per facilitare lo scambio significativo di dati tra macchine. Quando i motori di ricerca trovano e indicizzano contenuti dal web, la maggior parte dei dati nelle pagine web è non strutturata. La specifica HTML da sola non definisce un vocabolario condiviso che permetta di descrivere gli elementi su una pagina in modo standard e non ambiguo.

I Linked Data sono un insieme best practice che permettono alle risorse web di essere interconnesse in modo che i computer possano comprendere automaticamente il tipo e i dati di ciascuna risorsa.

JSON- LD

JSON-LD offre un modo semplice per arricchire semanticamente i documenti JSON aggiungendo informazioni contestuali e collegamenti ipertestuali per descrivere i diversi elementi degli oggetti JSON. È principalmente pensato per utilizzare Linked Data in ambienti di programmazione basati sul web, costruire servizi web interoperabili e memorizzare Linked Data in motori di archiviazione basati su JSON.

Shared Knowledge:

La conoscenza può essere rappresentata in formati di dati strutturati:

- **Vocabolario:** Un elenco di termini (entità), ciascuno con una definizione e i loro tipi, che diventa un insieme di URI. I vocabolari definiscono i concetti e le relazioni usati per descrivere e rappresentare un'area di interesse.
- **Grafo:** Una rete in cui i termini (entità) sono collegati da proprietà, ciascuna con una definizione, che diventa un altro insieme di URI. I grafi rappresentano i dati come nodi (entità) e archi (relazioni), formando una struttura di dati flessibile e interconnessa.
- **Ontologia:** Una specifica formale di un insieme di concetti all'interno di un dominio e le relazioni tra questi concetti. Include entità, tipi, proprietà e un insieme di regole che definiscono la conoscenza per un dominio specifico, creando un quadro strutturato per comprendere le relazioni all'interno dei dati.

Schema.org:

Schema.org è un progetto collaborativo e guidato dalla comunità che fornisce una raccolta di schemi condivisi per dati strutturati su Internet. Il suo scopo è standardizzare il markup dei dati per migliorare l'ottimizzazione per i motori di ricerca, facilitare l'integrazione dei dati tra diverse piattaforme e migliorare l'usabilità e la chiarezza dei dati web.

Grafi di conoscenza:

Un Knowledge Graph è una rappresentazione strutturata delle conoscenze e delle relazioni tra entità nel mondo reale. È una forma di database che utilizza un modello di grafo per organizzare le informazioni, dove i nodi rappresentano le entità e i collegamenti tra i nodi rappresentano le relazioni tra di esse. Questo modello permette di catturare complessi contesti e connessioni tra le informazioni, consentendo alle macchine di comprendere meglio il significato dei dati e di fornire risultati di ricerca più pertinenti e contestualizzati.

Resource Description Framework:

L'RFD è un modello dati per rappresentare informazioni sul web. Si basa su concetti fondamentali come le "triple" che sono l'unità base per organizzare le informazioni e consistono in soggetto, predicato e oggetto; su grafi diretti (etichettati), che sono insiemi di triple e su URI, identificatori univoci di risorse, che consentono di identificare in modo univoco le risorse nel web.

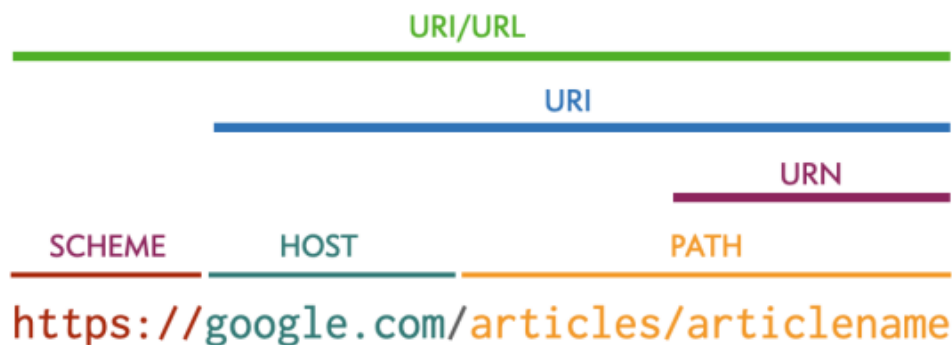
RDF utilizza una sintassi XML soggetta a restrizioni specifiche da parte di un DTD (Document Type Definition). Le dichiarazioni sono strutturate in forma di "soggetto/predicato/oggetto" e si utilizza l'URI per identificare ogni cosa di cui si vuole parlare, che può essere qualsiasi risorsa come un pezzo di informazione, un documento o un concetto in un'ontologia particolare.

Uniform Resource identifier:

L'URI è una stringa di caratteri utilizzata per identificare un nome o una risorsa.

Esistono diversi tipi di URI:

- **URL (Uniform Resource Locator):** un sottoinsieme dell'URI utilizzato per identificare le risorse su Internet tramite un indirizzo web.
- **URNs (Uniform Resource Names):** sono pensati per essere identificatori unici e persistenti, indipendentemente dal luogo o dalla modalità di accesso alla risorsa e non implicano necessariamente la disponibilità immediata o l'accesso alla risorsa identificata.



Da URI a IRI:

Un IRI (Internationalized Resource Identifier) è un'estensione degli URI che consente di utilizzare un set più ampio di caratteri, inclusi quelli non ASCII, per identificare risorse su internet in modo globale. La transizione da un URI a un IRI comporta principalmente l'uso di caratteri non ASCII, come ad esempio lettere accentate, caratteri cinesi, giapponesi o arabi, all'interno dell'identificatore della risorsa.

Cloud Computing:

Internet of things (IOT):

L'Internet of things è un sistema di oggetti fisici che possono essere scoperti, monitorati, controllati da dispositivi elettronici che comunicano tramite diverse interfacce di rete e possono essere connessi a Internet. La sfida è costruire un "ecosistema" di servizi integrati, utilizzando il Web come piattaforma programmabile, gemelli digitali per includere oggetti fisici e il cloud computing come paradigma per costruire sistemi di sistemi.

Cloud computing:

Il cloud computing è uno stile di computazione che fornisce capacità "IT" scalabili ed elastiche come servizio a clienti esterni utilizzando tecnologie internet. Inoltre, è una metodologia di gestione dell'infrastruttura che consente di gestire un gran numero di risorse altamente virtualizzate che possono essere automaticamente aggregate per offrire servizi.

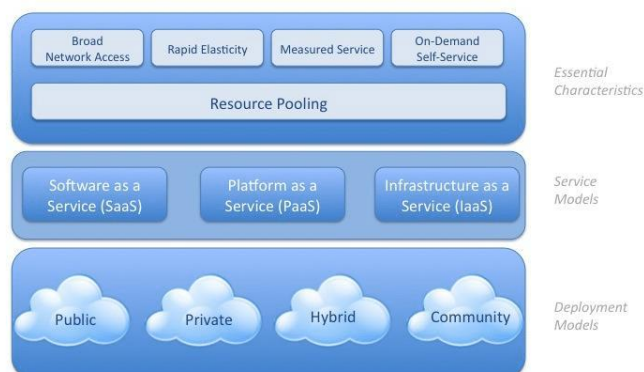
Cloud:

Il cloud è un modello che consente un accesso di rete ubiquitario, conveniente e su richiesta a un pool condiviso di risorse informatiche configurabili (ad esempio, reti, server, storage, applicazioni e servizi) che possono essere rapidamente fornite e rilasciate con uno sforzo minimo di gestione o interazione con il fornitore di servizi.

Questo modello promuove la disponibilità ed è composto da **cinque caratteristiche essenziali, tre modelli di servizio e quattro modelli di deployment**.

Caratteristiche del cloud:

- **On demand self-service:** l'utente può accedere alle risorse ed utilizzare i servizi senza entrare in contatto con il cloud provider.
- **Broad network access:** i servizi sono disponibili tramite la rete e accessibili attraverso meccanismi standard e possono essere utilizzati da piattaforme client differenti.
- **Resource pooling:** le risorse informatiche del fornitore sono condivise tra vari consumatori usando un modello multi-tenant, con risorse fisiche e virtuali assegnate e riassegnate dinamicamente secondo la domanda.
- **Rapid elasticity:** l'utente può richiedere l'aumento o la diminuzione delle risorse in maniera elastica.
- **Measured service:** i sistemi cloud controllano e ottimizzano automaticamente l'uso delle risorse tramite capacità di misurazione, fornendo trasparenza per entrambi, il fornitore e il consumatore del servizio.



Modelli di deployment:

Private cloud:

L'infrastruttura cloud è operata esclusivamente per un'organizzazione. Può essere gestita dall'organizzazione stessa o da un terzo e può esistere in loco o fuori sede. Anche conosciuto come cloud interno o cloud on-premise, un cloud privato limita intenzionalmente l'accesso alle sue risorse ai consumatori di servizi che appartengono alla stessa organizzazione che possiede il cloud.

Public cloud:

L'infrastruttura cloud è resa disponibile al pubblico generale o a un grande gruppo industriale ed è di proprietà di un'organizzazione che vende servizi cloud.

Hybrid cloud:

Il cloud ibrido combina elementi del cloud pubblico e privato, permettendo alle organizzazioni di sfruttare i vantaggi di entrambi i modelli. Questo modello offre una grande flessibilità, poiché le risorse possono essere facilmente spostate tra ambienti pubblici e privati in base alle esigenze. Il cloud ibrido offre anche l'opportunità di ottimizzare i costi, utilizzando il cloud pubblico per risorse temporanee o non critiche e il cloud privato per risorse più sensibili.

Community cloud:

Il cloud comunitario è un modello di deployment in cui diverse organizzazioni con requisiti simili condividono un'infrastruttura di cloud computing. Questo modello è particolarmente utile per le organizzazioni che hanno esigenze simili in termini di privacy, conformità o prestazioni.

Modelli di servizi:

Ci sono tre tipi di servizi:

- Software As a Service (SAAS);
- Platform As a Service (PAAS);
- Infrastructure As a Service (IAAS).

Virtualizzazione:

La funzionalità più importante della virtualizzazione è la capacità di eseguire contemporaneamente più sistemi operativi e applicazioni, indipendentemente dalla piattaforma o dall'hardware sottostante.

La virtualizzazione consente di:

- Isolare fallimenti causati da errori o problemi di sicurezza;
- Introdurre nuove capacità di sistema senza aggiungere complessità all'hardware e al software già esistenti;
- Migliorare le prestazioni complessive delle applicazioni grazie alla tecnologia che bilancia le risorse e fornisce solo ciò di cui l'utente necessita.

Livelli di interfaccia:

- **Istruzioni macchina non privilegiate:** sono un'interfaccia per l'hardware di base disponibile per qualsiasi programma;

- **Istruzioni macchina privilegiate:** forniscono un'interfaccia per l'hardware per il sistema operativo;
- **Chiamate di sistema:** forniscono un'interfaccia al sistema operativo per le applicazioni;
- **API:** sono un'interfaccia del sistema operativo attraverso chiamate di funzione.

Tipi di virtualizzazione:

Process virtual machine

Il Process Virtual Machine è una forma di virtualizzazione attraverso interpretazione o emulazione, tipicamente dedicata a un unico processo. Fornisce un set di istruzioni di macchina astratte; i programmi vengono compilati in codice macchina" che viene quindi interpretato o emulato.

Virtual machine monitor

Il Virtual Machine Monitor è in grado di fornire una macchina virtuale a molti programmi diversi contemporaneamente, come se più CPU fossero in esecuzione su una singola piattaforma. È particolarmente importante per i sistemi distribuiti perché le applicazioni sono isolate e gli errori sono limitati a una singola macchina virtuale.

Microservizi:

Il concetto di architettura a microservizi suddivide ogni elemento di funzionalità in un servizio separato, rendendo ciascun servizio un'unità indipendente di distribuzione.

La Microservices Architecture pattern superficialmente assomiglia all'Approccio SOA. In entrambi i casi, l'architettura consiste in un insieme di servizi, tuttavia le applicazioni basate su microservizi preferiscono protocolli più semplici e leggeri come REST.

I microservizi sono basati sul principio della singola responsabilità: ogni modulo o classe deve avere la responsabilità rispetto ad una singola funzionalità fornita dal software, e questa responsabilità dovrebbe esser totalmente incapsulata dalla classe. Dovrebbero anche minimizzare le comunicazioni e la coordinazione tra persone, riducendo quindi i rischi.

Applicazioni monolitiche:

Nel cuore dell'applicazione c'è la logica di business, che è implementata in moduli che definiscono servizi, dominio di oggetti ed eventi. Intorno al cuore ci sono gli adapters, delle interfacce che includono l'accesso al database, il dialogo con i componenti e l'esposizione delle API.

Anche se logicamente è un'architettura modulare, l'applicazione è impacchettata e distribuita come un monolite

Questo tipo di architettura se implementata su larga scala porta a diversi problemi:

- Difficoltà di manutenzione ed evoluzione.
- L'aggiunta o l'aggiornamento di librerie porta a sistemi inconsistenti.
- Riavvio obbligato per qualsiasi modifica in un modulo.
- Possono soffrire di problemi di affidabilità: un bug in qualsiasi modulo, come una perdita di memoria, può potenzialmente far cadere l'intero processo.
- Hanno costi di distribuzione elevati rispetto ai singoli moduli.

Architettura dei microservizi:

Invece di costruire un'unica applicazione monolitica e mostruosa, l'idea è suddividere l'applicazione in un insieme di servizi più piccoli e interconnessi. Un servizio implementa un insieme di funzionalità distinte. Ogni microservizio è a tutti gli effetti una mini-applicazione.

I dati:

Il pattern di architettura a microservizi ha un impatto significativo sulla relazione tra l'applicazione e il database. Ogni servizio ha il proprio schema del database. Questo approccio contrasta con l'idea di un modello di dati aziendale, spesso risultando nella duplicazione di alcuni dati. Tuttavia, garantisce un accoppiamento lento, permettendo ai servizi di essere sviluppati, testati e distribuiti in modo indipendente.

Benefici dei microservizi:

L'architettura a microservizi affronta il problema della complessità suddividendo l'applicazione in blocchi o servizi gestibili. Ogni servizio ha un confine ben definito sotto forma di un'API basata su RPC o su messaggi. I singoli servizi sono molto più veloci da sviluppare e molto più facili da comprendere e mantenere. Questo approccio consente a ciascun servizio di essere sviluppato indipendentemente.

L'architettura a microservizi consente a ciascun microservizio di essere distribuito indipendentemente, ciò rende possibile il rilascio continuo: gli sviluppatori non devono coordinare la distribuzione di modifiche relative al loro servizio. Questi tipi di cambiamenti possono essere distribuiti non appena sono stati testati.

Inoltre, l'architettura a microservizi consente a ciascun servizio di essere scalato indipendentemente. È possibile distribuire il numero di istanze di ciascun servizio che soddisfano i suoi vincoli di capacità e disponibilità. Inoltre, è possibile utilizzare l'hardware che meglio corrisponde ai requisiti di risorse di un servizio.

Dimensioni di scalabilità:

- **X-axis** esegue molteplici copie di una stessa applicazione con un load balancer;
- **Y-axis** suddivide l'applicazione in differenti servizi multipli;
- **Z-axis** ogni server è responsabile per un solo subset di dati.

L'architettura a microservizi opera sull'asse Y.

Containers:

Programmable platforms:

I componenti funzionali potrebbero necessitare di componenti aggiuntivi per fornire le proprietà e le qualità richieste. Applicazioni, librerie e strumenti dovrebbero essere distribuiti insieme per semplificare la gestione e garantire un comportamento coerente, ad esempio, dall'infrastruttura di sviluppo a quella di produzione.

Container:

Il container è un modo standard per impacchettare un'applicazione e tutte le sue dipendenze, in modo che possa essere spostata tra ambienti ed eseguita senza modifiche.

Utilizzi dei container:

I container aiutano a rendere il flusso di lavoro di sviluppo e build locale più veloce, efficiente e leggero, consentono l'esecuzione di servizi e applicazioni stand-alone in modo coerente su più ambienti. Inoltre, consentono la costruzione e il test di applicazioni complesse e architetture su un host locale prima della distribuzione in un ambiente di produzione. Facilitano la costruzione di infrastrutture PaaS multiutente. Offrono ambienti sandbox leggeri e stand-alone per lo sviluppo, il test e l'insegnamento di tecnologie come Unix Shell o un linguaggio di programmazione. Supportano applicazioni SaaS e distribuzioni altamente performanti e iper-scalabili di host.

Docker:

Docker è una piattaforma aperta per costruire applicazioni distribuite. Alla base di Docker ci sono:

- **Image:** un read-only snapshot di un container che può essere usato come template per costruire un container.
- **Container:** l'unità di base in cui le applicazioni risiedono e sono trasportate.
- **Docker Hub/Registry:** è disponibile come SaaS per il deployment, immagazzina, distribuisce e condivide immagini per container.
- **Docker engine:** un programma che crea ed esegue container può essere eseguito sia su macchine fisiche che su macchine virtuali, sia su un cloud privato che su un cloud pubblico. Il client comunica con la piattaforma attraverso l'esecuzione di comandi.

Docker engine:

Docker Engine è un'applicazione client-server composta da un server, chiamato processo demone, una REST API che definisce le interfacce per la comunicazione con il demone e un client della riga di comando.

Docker image:

Un'immagine Docker è composta da file system stratificati uno sopra l'altro. Alla base c'è un file system di avvio. Utilizza la tecnica del mount union, che consente a diversi file system di essere montati contemporaneamente ma di apparire come un unico file system. Il pattern "copy on write" è una delle caratteristiche che rende Docker così potente. Quando Docker avvia per la prima volta un container, il layer iniziale di lettura-scrittura è vuoto. Man mano che si verificano cambiamenti, questi vengono applicati a questo layer; ad esempio, se si desidera modificare un file, questo viene copiato dal layer di sola lettura sottostante al layer di lettura-scrittura. La versione di sola lettura del file continuerà a esistere, ma sarà nascosta sotto la copia.

Architettura Docker:

Docker utilizza un'architettura client-server, in cui il client Docker e il demone possono essere eseguiti sullo stesso sistema o connessi a distanza. Il client e il demone comunicano tramite una REST API, utilizzando socket UNIX o un'interfaccia di rete. Le principali operazioni includono:

- **Build:** utilizzata per creare un'immagine (un'applicazione);
- **Pull:** per recuperare un'immagine da un registry;
- **Run:** per eseguire un'istanza di un'immagine.

Un registry Docker memorizza le immagini Docker; Docker Hub e Docker Cloud sono registry pubblici e Docker cerca immagini su Docker Hub per impostazione predefinita.